SimFoundry: Modular and Automated Scene Generation for Policy Learning and Evaluation

Nadun Ranawaka[1,2*], **Josiah Wong**[1,3*], **Wei-Lin Pai**[3], **Wei-Teng Chu**[3], **Tianyuan Dai**[1,4], **Masoud Moghani**[1,5], **Hang Yin**[3], **Yunfan Jiang**[1,3], **Wesley Durbano**[1*], **Brandon Huynh**[1*], **Yu Fang**[1], **Linxi Fan**[1], **Danfei Xu**[1,2], **Ruohan Zhang**[3], **Li Fei-Fei**[3], **Bowen Wen**[1], **Ajay Mandlekar**[1†], **Yuke Zhu**[1,4†]

[1]NVIDIA [2]Georgia Institute of Technology [3]Stanford University [4]The University of Texas at Austin [5]University of Toronto
*Equal contribution †Equal advising

arXiv:2606.28276v1 [cs.RO] 26 Jun 2026

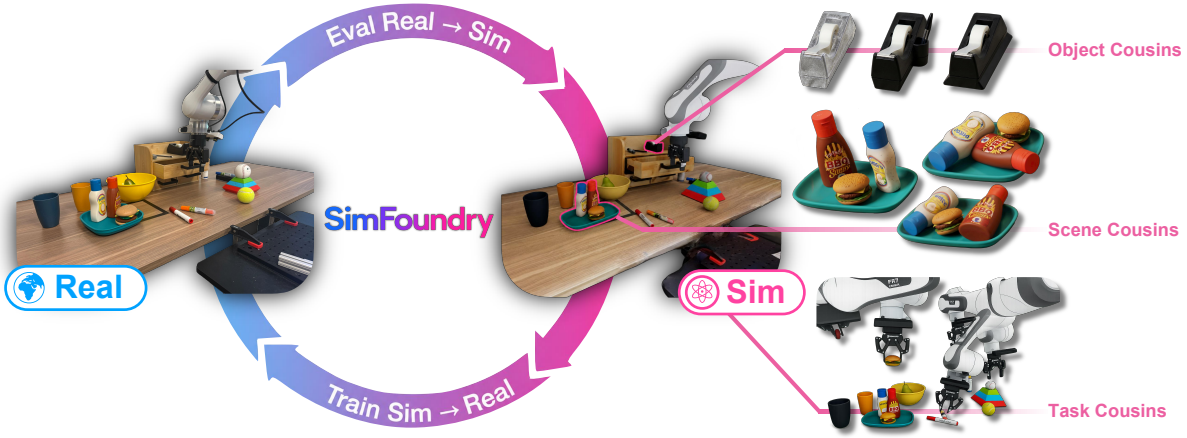


Figure 1: **Overview.** SimFoundry takes a single real-world input video and automatically reconstructs an interactive, sim-ready digital twin of the scene. Based on the reconstructed digital twin, SimFoundry can further generate an unlimited number of *digital cousins* — affordance-preserving variants of the original scene, spanning three different axes of variation, which we term *object*, *scene*, and *task cousins*, respectively. These generated simulation environments support both real-to-sim policy evaluation and sim-to-real policy training, enabling policies to be benchmarked and improved at scale before deployment in the real world.

Abstract

Training and evaluating robot policies in the real world is costly and difficult to scale. We introduce SimFoundry, a modular and automated system for zero-shot real-to-sim scene construction from a video. SimFoundry generates sim-ready digital twins and supports object, scene, and task editing, enabling the automated generation of diverse digital cousins: affordance-preserving variations of reconstructed real-world scenes. Policies trained on SimFoundry data transfer zero-shot to challenging real tasks involving multi-step manipulation, articulated object interaction, and bimanual interaction, and its digital cousins (variations of the original scene and objects) facilitate generalization to new real-world conditions. Across 7 manipulation tasks and 5 policy architectures, SimFoundry simulation evaluations strongly predict real-world performance, with mean Pearson correlation 0.911 and mean maximum ranking violation 0.018. When evaluating sim-trained policies zero-shot in the real world, policies trained with object, scene, and task cousins in simulation show average task success rate improvements of 17%, 21%, and 40%, respectively. Additional details at `https://research.nvidia.com/labs/gear/simfoundry/`.

Keywords: Real2Sim, Sim2Real, Scene Generation, Policy Learning, Policy Evaluation

© 2026 NVIDIA. All rights reserved.

1. Introduction

Robotic foundation models [5, 61] trained on large-scale robot manipulation datasets have enabled robots to perform a wide range of manipulation tasks autonomously. However, sourcing high-quality robot manipulation data in large volumes is labor-intensive, often involving large-scale robot teleoperation efforts spanning many months or years [5, 6, 20, 39]. Moreover, evaluating trained foundation models in a systematic and scientific manner on real-world manipulation problems of interest can be costly and require thousands of trials across different tasks to make rigorous comparisons [4].

In response to these bottlenecks, recent work has explored simulation as a scalable alternative for training and evaluating robot manipulation models. Automated data generation tools can synthesize large volumes of diverse, high-quality demonstrations with minimal human effort [18, 23, 37, 46, 58], and have been used to train and improve real-world agents [13, 26, 55, 86]. Recent work has also shown that simulation-based evaluations can strongly correlate with real-world results, offering a time- and cost-efficient alternative to physical benchmarking [3, 47]. However, manually constructing simulation environments remains challenging, especially when they must align with real-world scenes and tasks in visuals, geometry, and dynamics.

To address the issues of manual simulation development, real-to-sim scene construction [19, 79] has emerged as a paradigm for generating synthetic scenes grounded in the real world. By leveraging 3D reconstruction and generative models, users can create "sim-ready" environments that support physically grounded robotic interaction with minimal manual effort. While this reduces environment-authoring overhead and enables both sim-to-real transfer [17, 19, 27, 34, 79, 105] and predictive real-world benchmarking [32, 33, 104], few works provide a unified system that automates scene reconstruction while also performing real-to-sim policy evaluation and training policies that transfer across domains. Recent systems that primarily generate simulation-ready 3D scenes [63, 85, 96] often lack the physical interaction, task specification, or data-generation machinery needed to close the sim-to-real policy-learning loop. Conversely, systems designed for simulation-based policy evaluation often assume manually tuned scenes, focus on short-horizon atomic manipulation, or do not support automatically generating diverse objects, scenes, and tasks for policy training [27, 32, 34, 40, 101].

To this end, we introduce SimFoundry, a unified and modular system that turns a single real-world input video into interactive simulation environments for both policy evaluation and policy training. SimFoundry unifies three capabilities that are often treated separately in prior work: reconstructing a sim-ready digital twin, expanding that reconstruction into diverse training environments, and using the resulting simulations to both benchmark and train robot policies. Its modular design decomposes real-to-sim construction into interchangeable components for perception, asset generation, pose alignment, articulation, physics annotation, and data generation, allowing improved foundation models to be incorporated as they become available without redesigning the full system. To scale beyond a single reconstructed scene, SimFoundry automatically generates *digital cousins*: affordance-preserving variations that maintain the task-relevant semantics of the original scene while varying object instances, spatial layouts, and feasible manipulation tasks. These digital cousins, spanning object-, scene-, and task-level variations, enable large-scale synthetic data generation for training robot policies that are robust to changes in object morphology and scene layout and that generalize to related manipulation tasks. SimFoundry environments also produce simulation evaluations that strongly correlate with real-world policy performance. Finally, policies trained on SimFoundry simulation trajectories can successfully deploy in their real-world counterparts.

Summary of Contributions:

• We introduce SimFoundry, a fully automated and modular real-to-sim system that generates interactive, sim-ready scenes from a single video. SimFoundry supports rigid and articulated objects, physics annotations, and automated object, scene, and task cousins, augmenting a single scene into diverse training environments. Across 12 reconstruction scenes, SimFoundry achieves zero-shot F1 scores of 0.81–0.92, which can be further improved to 0.93–0.99 with only 3 minutes of per-object tuning.

• We demonstrate SimFoundry on manipulation tasks that exceed prior real-to-sim work in manipulation complexity, including multi-step tasks, articulated-object interaction, and bimanual coordination on both DROID and YAM robot embodiments. Across 7 tasks and 5 policy types, SimFoundry simulation evaluations

Table 1: **System comparison.** SimFoundry provides a unified and modular pipeline for real-to-sim scene generation that is more feature complete than prior works.

	ACDC [17]	RialTo [79]	DRAWER [95]	RoLA [105]	R2R2R [102]	SIMPLER [47]	PolaRiS [32]	RobotArena∞ [33]	R2S-Soft [104]	Re^{3}Sim [27]	GSWorld [34]	SAGE [96]	MolmoSpaces [40]	GenieSim [101]	**SimFoundry (Ours)**
Sim-to-real training	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓	✓	✗	✓	✓	✓
Real-to-sim policy eval	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	✗	✓	✗	✓
Automatic scene construction	✓	✗	✓	✓	✗	✗	✗	✓	✓	✓	✓	✓	✗	✓	✓
Articulated objects	✓	✓	✓	✗	✓	✓	✗	✗	✗	✗	✗	✓	✓	✓	✓
Multi-embodiment	✗	✗	✗	✓	✗	✓	✗	✓	✗	✗	✓	✗	✓	✗	✓
Asset generation	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✗	✓	✓
Background reconstruction	✗	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓
Object cousins	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
Scene cousins	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
Task cousins	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓

strongly correlate with real-world performance with a mean Pearson correlation of 0.911 and an MMRV of 0.018, outperforming the state-of-the-art baseline [32] by over 0.59 on the Pearson correlation.

• We demonstrate that SimFoundry-generated data can train policies that transfer to the real world and generalize beyond the reconstructed twin. Policies trained on SimFoundry simulation data achieve strong, and sometimes near-perfect, real-world success rates on both YAM and DROID. Multi-task SimFoundry data further improves generalist policies by up to 31% in simulation and 18% in the real world, while reaching 29% success rate on held-out real tasks.

2. Related Work

3D Asset Generation and Alignment. 3D asset reconstruction and generation has evolved along multiple fronts. Retrieval-based methods align CAD models from databases to single-view images [2, 17, 22, 25, 41, 42]. In parallel, generative models synthesize high-fidelity meshes from one or a few images [10, 28, 75, 77, 93, 97, 98], and recent extensions expand to handle articulated objects with automatically inferred movable parts [11, 12, 36, 43, 48, 51, 81, 92, 99, 103]. Precise object alignment in multi-object scenes draws on vision foundation models for depth [50, 90, 91], segmentation [53, 67], and 6-DoF pose and scale estimation [44, 88, 89]. SimFoundry is inherently modular to compose these primitives, allowing newer tools to be swapped in and outputs to be refined via human interventions.

Real-to-Sim for Simulation Environment Creation and Applications. The advent of high-quality 3D reconstruction and generative 3D synthesis have unlocked several works that automate simulation environment construction from real-world captures [1, 17, 19, 27, 32–34, 36, 40, 49, 63, 66, 79, 82, 85, 94, 96, 104]. These systems broadly target one of two goals: closing the real-to-sim-to-real loop for agent training [14, 17, 21, 24, 78, 79, 95, 105], or providing reconstructed environments for reliable policy evaluation [3, 27, 32–34, 47, 101, 104]. A separate body of work sidesteps physical simulation entirely and uses reconstructed scenes only for rendering [102], which limits applicability to contact-rich robot tasks. SimFoundry belongs to a select group of real–to–sim systems [27, 34, 101] that demonstrate both successful sim-to-real agent transfer and strong correlation between simulated and physical policy evaluations. However, SimFoundry goes beyond these systems, in its ability to handle more diverse task characteristics (including bimanual, articulation, and multi–step manipulation) and its support for developing multiple types of cousins of a digitized real–world scene to scale up the diversity of reconstructed environments. Table 1 summarizes these differences and Appendix D provides a more detailed related work discussion.

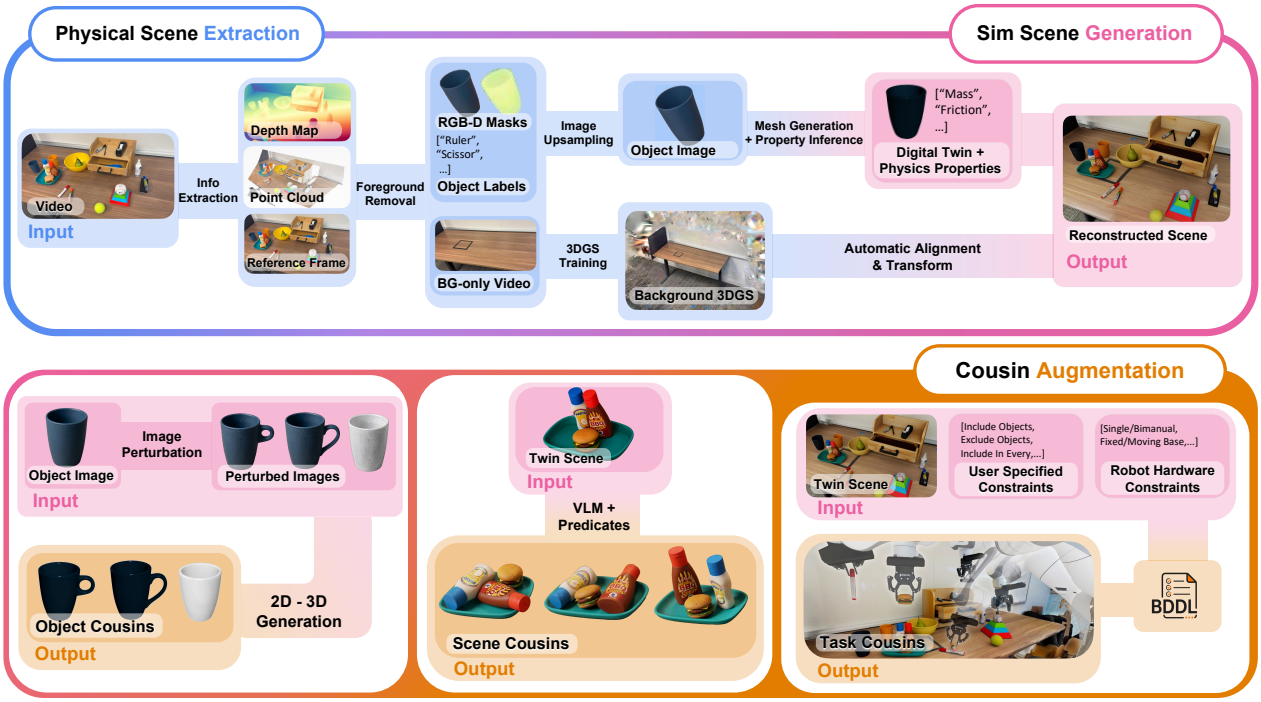


Figure 2: **Method Overview.** SimFoundry extracts per-object relevant information (segmentation masks, depth, etc.), generates 3D visual meshes via 2D-to-3D generation models, and compiles the final output scene by annotating relevant physical parameters and sanity checking the overall scene configuration in a physics simulator. SimFoundry additionally supports diverse simulated augmentations along these axes of variation on object, scene, and task: *object cousins* can be generated by modifying input objects in their image space and re-generating corresponding 3D meshes; *scene cousins* can augment the configuration of objects; and *task cousins* can propose viable interactions within the scene.

3. Preliminaries

Overview. In this work, we seek to apply SimFoundry to first reconstruct real world scenes $\mathcal{S}_{real}$ in simulation $\mathcal{S}_{sim}$ by converting an input video into a set of object meshes $\mathcal{M}_i$, scales $\mathbf{s}_i$, and poses $\mathbf{p}_i$, where $i \in \{1, \ldots, N\}$, leveraging multiple foundation models V_* to achieve this. We then apply SimFoundry to downstream robotics applications. We broadly define a policy π_θ mapping observations at the current timestep o_t to actions a_t, $\pi_\theta : \mathcal{O} \rightarrow \mathcal{A}$, implemented as a neural network parameterized by θ. We focus on two applications: **real-to-sim evaluation**, where existing real-world policies are evaluated in simulation to characterize their performance, and **sim-to-real training**, where policies are trained in simulation and then deployed in the real world.

Real and Simulation Policy Correlation. We measure real and sim policy correlation using the Pearson Correlation Coefficient (r) and Mean Maximum Rank Violation (MMRV), both of which have been proposed by prior work [32, 47]. Ideal correlation has $r \rightarrow 1$, which measures linear correlation between real and simulation task results, and MMRV $\rightarrow 0$, which measures the average worst rank-violation of policies as evaluated in simulation versus their actual ranks in the real world. When measuring task success, we measure end-to-end task success (a discrete 0 or 1), which is defined as the completion of all task criteria (see Appendix I.2).

Real-to-Sim Reconstruction and Synthetic Simulation Data. We define *digital twins* as being strict replicas of the geometry and object layouts of a real-world scene. In contrast, *digital cousins* [17, 55] are virtual scenes that maintain the semantic and geometric affordances of a real-world scene without explicitly modeling it, and serve as a form of object instance randomization. MimicGen [37] is a recent method proposed to quickly generate large amounts of synthetic trajectories by splicing together various subtask trajectories sampled from a set of source demonstrations.

4. SimFoundry: A modular, automated real-to-sim generation pipeline

SimFoundry generates interactive simulated scenes through three stages, as seen in Figure 2: **Extraction**, which infers relevant per-object information from a video; **Generation**, which creates, aligns, annotates, and stabilizes sim-ready assets; and **Augmentation**, which produces digital cousins in the form of object, scene, and task variations. Appendix E.2 has additional details, including the underlying foundation models used (denoted as V_*).

Extraction. We assume the input is a raw RGB video. We first convert the input into a representative RGB frame $\mathbf{I}_s$ and estimate a corresponding depth map $\mathbf{D}_s$ using off-the-shelf depth estimation models $V_{im2depth}$ [50, 90]. Using the camera intrinsics $\mathbf{K}$, we lift this RGB-D observation into a scene point cloud $\mathbf{P}_s$, which is used for scene alignment and object pose estimation. We then query V_{seg}^{image} [10] to extract the ground plane and align the reconstruction with the simulator world frame.

Next, we use a scene-understanding VLM V_{scene} to detect the objects in the scene and V_{seg}^{image} to iteratively segment out the foreground objects $o_1, \ldots, o_n$. For each object, we extract its segmentation mask m_i from V_{seg}^{image} along with corresponding RGB and depth pixels (p_i^{rgb}, p_i^{depth}). After each extraction, we remove the object from the RGB-D observation using image and depth inpainting, and repeat this process until no foreground objects remain. This stage outputs per-object RGB-D crops and masks, which are used for mesh generation and alignment. Further details on video preprocessing, inpainting, and iterative decomposition are provided in Appendix E.1.

Generation. Given each object crop p_i^{rgb}, we use V_{image} to upsample and a 2D-to-3D mesh model V_{mesh} to generate a visual mesh $\mathcal{M}_i$. We then estimate and refine the object pose $\mathbf{p}_i$ by aligning the mesh to the reconstructed scene using the scene RGB-D observation, object mask, and point cloud geometry, with additional refinement from another model V_{pose} such as FoundationPose [89]. Objects identified as articulated, such as cabinets or drawers, are processed by a separate articulation module that detects movable parts, segments the mesh, and generates joint parameters using $V_{articulation}$ and prior articulation-generation methods [43, 65]; full details are provided in Appendix E.3.

Finally, for each generated object, we produce collision geometry using CoACD [87] and assign physical properties such as mass and friction by querying V_{scene} . Once all objects are generated, aligned, and annotated, we compose the scene in PyBullet [16], resolve object penetrations to obtain a stable configuration, and export the resulting sim-ready scene to downstream robotics simulators such as IsaacLab [59]. Further details are listed in Appendix E.4.

Augmentation. Once the initial scene is reconstructed, SimFoundry expands it into a family of digital cousins: affordance-preserving simulated variants that retain its task-relevant semantics while varying three axes: object instance, scene layout, and task specification. Figure 3 illustrates this process across diverse real-world scenes. The middle row shows that SimFoundry reconstructs the objects, layouts, and scene structure of the real-world inputs, while the bottom row demonstrates how the reconstructed twins can be expanded into plausible digital cousins. These cousins alter object geometry, appearance, spatial configuration and task specifications and for brevity, we refer to digital cousins generated by varying these axes as *object cousins*, *scene cousins*, and *task cousins*, respectively. These terms indicate the dominant axis of variation rather than mutually exclusive classes: a digital cousin may combine object, scene, and task variations. Further details are provided in Appendix F.

Object cousins generate new object instances that maintain the affordances of the original object while varying geometry, topology, and appearance. For example, a reconstructed mug, drawer, or plate can be converted into multiple plausible alternatives with different shapes, handles, textures, or proportions. These object-level cousins provide instance diversity while preserving task-relevant functionality.

Scene cousins vary the spatial arrangement of objects in the reconstructed scene using semantic spatial predicates such as **OnTop** and **RightOf**. Rather than simply perturbing object poses, these cousins produce meaningful alternative layouts, such as moving an object from beside a receptacle to inside or on top of it. We can also add controllable distractor objects from a library of sim-ready assets. These scene-level cousins introduce structured geometric diversity and help policies generalize beyond the original layout.

Figure 3: **SimFoundry Scene Generation Samples.** We show real-world input images (top row), the corresponding reconstructed digital twins generated by SimFoundry (middle), and sampled digital cousin scene variations (bottom). For instance, in the scene that is second from the left, the brown glass bottle becomes narrower for the cousin scene, and in the scene that is third from the right, the digital cousin of the wicker basket has holes near the top that can plausibly be used as handles, while the layout of the scene also changes.

Task cousins use the reconstructed scene to propose additional feasible manipulation tasks grounded in the available objects and affordances. SimFoundry converts these tasks into simulation-compatible goal specifications, enabling procedural demonstration collection without manually authoring each environment or task. This allows the same reconstructed scene to support multi-task data generation, including related tasks that share objects, goal conditions, or intermediate behaviors with the original task.

Together, these mechanisms provide controllable diversity across objects, layouts, and tasks. In our experiments, object cousins improve robustness to unseen object instances, scene cousins improve generalization to novel layouts, and task cousins improve both zero-shot and few-shot downstream task performance.

Background Reconstruction and Alignment. The Extraction-Generation-Augmentation pipeline produces a physically-grounded foreground scene of per-object meshes. To recover a photorealistic background, SimFoundry can fuse reconstructed objects with a 3D Gaussian Splat [38] background. We support two pipelines to this end. The *automatic* pipeline operates on the same single raw video used by the Extraction stage: it removes foreground objects via prompted video segmentation and two-pass inpainting, recovers metric depth and camera poses, trains a depth-supervised splat, and bridges it into the simulator world through a derived rigid transform, requiring no additional capture or user input. The *manual* pipeline instead needs the user to take a second video of the scene with foreground objects physically removed. It then trains a splat on this background-only video, and aligns it to the reconstructed scene through our interactive editor. The two routes emit identical asset structures and, as we show in Appendix L.2, yield comparable reconstruction quality; they trade off capture effort against compute and fidelity on texture-less surfaces and silhouettes. We detail both pipelines and their respective strengths in Appendix E.5. We also use mesh background reconstruction, generated by apps such as Scaniverse[1], in our robotics experiments.

5. Experiments

We highlight two key applications of SimFoundry — using SimFoundry environments as a way to benchmark real-world manipulation policies (Sec. 5.1) and training robot manipulation agents from generated SimFoundry environments that transfer zero-shot to the real-world (Sec. 5.2). Sec. 5.3 contains additional experiments that analyze SimFoundry reconstruction performance. Our experiments are performed on two robot embodiments — the DROID [39] platform, and a YAM workcell [69]. The tasks we evaluate (shown in Figure 4) span diverse characteristics, including short-horizon pick and place, bimanual coordination, and long-horizon language following (see Appendix I for details).

[1] https://dev.scaniverse.com/

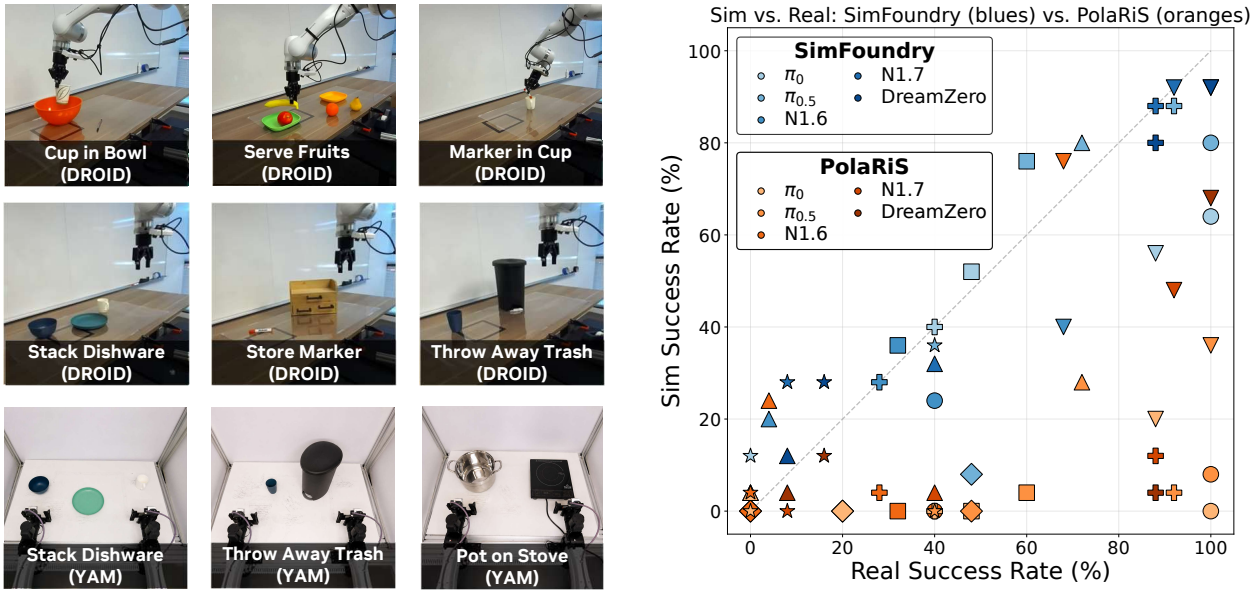


Figure 4: **Tasks and Real-to-Sim Policy Evaluation correlations.** (Left) We apply SimFoundry to a DROID setup using a single Franka arm (top two rows), and a bimanual setup with two YAM arms (bottom row). Our tasks span multiple types of manipulation, including multi-step, articulated object interaction, and bimanual coordination (**Clear Table** not shown, more details in Appendix I). (Right) SimFoundry outperforms the state-of-the-art baseline PolaRiS [32] in simulation-based evaluation correlations. Each marker shape represents a different task from the left. Additional details in Appendix G and Figure G.1.

5.1. Real-to-Sim Policy Evaluation

Setup. We aim to show that policy evaluations in SimFoundry simulation scenes can correlate strongly with real-world policy evaluation results. We consider two sets of policies and tasks — pre-trained generalist policies (π_0 [5], $\pi_{0.5}$ [31], GR00T N1.6 [61], GR00T N1.7, and DreamZero [100]) that are deployed zero-shot (no finetuning) on 4 less difficult tasks, and policies that are finetuned (π_0, $\pi_{0.5}$, and GR00T N1.6) using 50 real-world demos per task and deployed on 3 more challenging tasks. We use separate task sets for each policy group to ensure meaningful (non-zero) policy evaluation results for zero-shot and few-shot deployment. Appendix H contains details on policy training and selection, and Appendix J contains the evaluation procedure.

SimFoundry scene evaluations strongly correlate with real-world performance across diverse policies. As shown in Figure 4, SimFoundry evaluations closely match real-world results and preserve policy rankings, with a mean Pearson correlation of 0.911 and MMRV of 0.018 (Table G.1). SimFoundry evaluations also reveal model strengths: GR00T N1.7 can outperform others on precise grasping (e.g., **Marker in Cup**), while $\pi_{0.5}$ shows stronger language following (e.g., **Serve Fruits**), offering actionable guidance for model development. These correlations hold across policy types, including two VLA families (π and GR00T) and the world-action model DreamZero.

Sub-task evaluations improve correlations, especially on multi-step tasks. We introduce a sub-task evaluation procedure that increases policy eval correlations from a mean Pearson score of 0.90 to 0.95. By evaluating at a sub-task level, users can more accurately target future data collection for model improvement by focusing on specific sub-tasks. This procedure also helps improve eval correlations for long-horizon tasks, where success can be bottlenecked by a few, more difficult sub-tasks. Full details on the evaluation procedure and results are in Appendix G.1.1.

SimFoundry outperforms state-of-the-art simulation evaluation frameworks and makes fewer assumptions. We compare SimFoundry to PolaRiS [32], a state-of-the-art method for real-to-sim policy evaluation

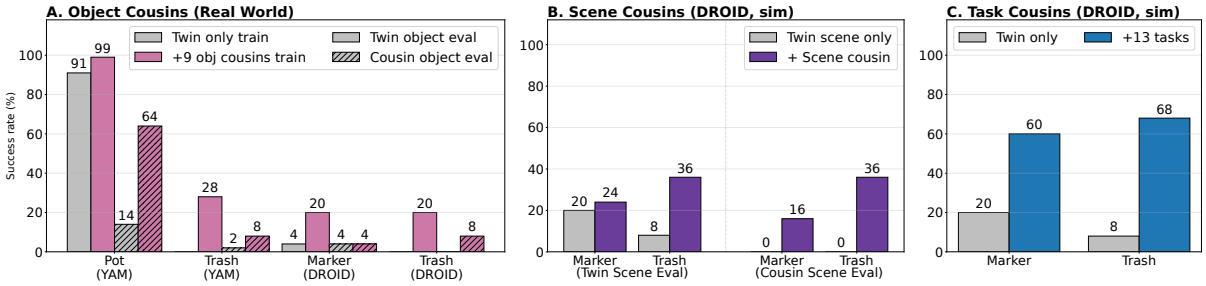


Figure 5: **SimFoundry Data Diversity Improves Policy Performance.** (A) Across multiple robot embodiments and multiple tasks, leveraging additional object cousins [17] improves direct Sim-to-Real policy transfer on the original target scene objects and additional held-out unseen objects. (B) Scene cousins improve policy performance on the original scene and allow policy transfer to cousin scenes. (C) Adding task cousins improves performance on related downstream tasks by enabling intra-task transfer. **Note:** Pot refers to the **Pot on Stove** task, Trash refers to the **Throw Away Trash** task and Marker is for the **Store Marker** task.

(full details of comparison in Appendix J.3). We use the same protocol to evaluate the same real-world policies in PolaRiS, and find that SimFoundry has a mean Pearson correlation that is over 0.59 higher than PolaRiS.

5.2. Sim-to-Real Policy Training

We show that SimFoundry can generate synthetic data for training policies that can be deployed in the real world. We study three settings: zero-shot sim-to-real transfer (policies trained only on SimFoundry data), sim-and-real co-training (policies trained on large-scale SimFoundry data plus limited real data), and multi-task transfer (policies trained on multi-task SimFoundry data that transfer to new real-world tasks). We make use of SimFoundry's ability to produce automated object, scene, and task cousins, and show that they are critical to enable real-world policy generalization. A summary of our results is given in Figure 5, while detailed results per experiment are given in Appendix G.

Policies trained on SimFoundry-generated data transfer zero-shot to the real world. Across both YAM and DROID, policies trained on SimFoundry data transfer effectively to real scenes, reaching 99% success on **Pot on Stove** with YAM and 100% success on **Stack Dishware** with DROID (Table G.4). On YAM, this is achieved with a simple flow-matching policy trained from scratch; on DROID, we finetune the $\pi_{0.5}$ [31] DROID checkpoint on sim-generated demonstrations.

Co-training with sim and real data further improves performance. Although SimFoundry supports zero-shot transfer, adding small amounts of real data further boosts performance. On DROID, co-training improves most π_0 and $\pi_{0.5}$ results in both sim and real (Figure G.2). For example, $\pi_{0.5}$ real-world success on **Store Marker** increases from 60% to 92%, while π_0 gains 36% sim success on **Throw Away Trash**. This suggests that SimFoundry reconstructions are faithful enough to complement real demonstrations during training.

Object cousins improve robustness to unseen objects. Across both embodiments, increasing object diversity improves policy performance. Adding object cousins yields a 50-point real-world gain on held-out **Pot on Stove** objects, and improves DROID performance in both sim and real with gains up to 20 points on **Throw Away Trash**. These results show that SimFoundry generates useful object-level diversity for training policies that generalize beyond the reconstructed twin. Additional details and ablations are provided in Appendix G.

Scene cousins improve layout generalization. On DROID, adding scene cousins boosts success in simulation by up to 28 points on **Throw Away Trash** in the twin scene (Figure 5B). Scene cousins also enable transfer to novel layouts, reaching 16% success on **Store Marker** cousin scenes where the twin-only policy achieves 0%.

Multi-Task Sim-to-Real and Task Generalization. We reconstruct a cluttered scene, use V_{scene} to propose tasks, generate demonstrations entirely in simulation, and fine-tune both base $\pi_{0.5}$ and $\pi_{0.5}$-DROID. We evaluate on 13 generated-data tasks and 7 held-out tasks in simulation and the real world. Results are presented in Table 2.

Table 2: **Multi-task policy evaluation.** Success rates for policies evaluated on seen and held-out tasks in simulation and the real world.

	$\pi_{0.5}$-DROID	$\pi_{0.5}$-FT	$\pi_{0.5}$-DROID-FT
Sim	30	51	**61**
Sim – held out	37	**45**	33
Real	28	45	**46**
Real – held out	26	**29**	26

SimFoundry can train generalist policies and task cousins improve few-shot downstream learning. SimFoundry-finetuned policies outperform the base DROID checkpoint by up to 31% in simulation and 18% in the real world, and $\pi_{0.5}$-FT reaches 29% success on held-out tasks without task-specific demonstrations (Table 2). With the total number of demonstrations fixed, replacing some target-task data with related task-cousin demonstrations improves downstream simulation performance, especially on harder tasks: in simulation, 13 task cousins increase success on **Throw Away Trash** by 60% and **Store Marker** by 40% (Figure 5C).

5.3. System Analysis

We analyze SimFoundry along three axes: reconstruction fidelity, the human effort required to refine that fidelity, and the scalability of the end-to-end pipeline (see Appendix L.1 for full details).

SimFoundry scene reconstruction fidelity outperforms state-of-the-art methods. Under fully automated operation, SimFoundry surpasses SAM3D across three 3D geometric metrics (Table L.2). For instance, SimFoundry achieves higher F1 score (0.81–0.92) than SAM3D (0.66–0.71), lower chamfer distance and position error, showing that the pipeline recovers precise scene geometry without human input.

SimFoundry environment generation scales well with compute and human time. The pipeline reconstructs objects at an average rate of roughly 5 minutes per object across diverse real-world scenes (Table L.4), and an additional 3 minutes of per-object operator tuning yields consistent gains on every metric (e.g. F1 scores rise to 0.93–0.99, as shown in Table L.2), demonstrating that fidelity can be traded against effort on demand.

6. Limitations

Our system relies heavily upon off-the-shelf foundation models. While this enables broad modularity and the ability to swap components as better models become available, it also naturally inherits the failure modes of each underlying model. Additionally, we make several assumptions that limit our current pipeline to tabletop-style layouts. Relaxing this assumption to support multi-level or non-planar environments is a natural direction for future work. A more thorough discussion of limitations is given in Appendix C.

7. Conclusion

SimFoundry is a fully automated pipeline that natively reconstructs interactive sim-ready scenes from a single video, handles articulated object generation and scenes with clutter and occlusion, and generates object, scene, and task cousins. We find that SimFoundry can measure task success in simulation that correlates to real-world policy performance and outperforms prior work in sim-based policy evaluations. Second, SimFoundry-generated data can train policies that transfer to the real world, while object, scene, and task cousins improve robustness to unseen objects, novel layouts, and related downstream tasks. SimFoundry enables simulation to accelerate real-world policy development by facilitating synthetic data generation for training policies and reliable simulation-based evaluation to compare policies and yield actionable insights.

8. Acknowledgments

The authors would like to thank Omkaar Buddhikot, Amitoj Sandhu, Nadia Laswi, Ramanpreet Singh, Mona Abbas, and Osiriz Durana for their help with data collection and model evaluation. We also thank Jeremy Chimienti, Danyi Chen, and Lion Park for their help with hardware support, and Scott Reed, You Liang Tan, and Fengyuan Hu for feedback and valuable discussions. Nadun Ranawaka is partially supported by the Agricultural Technology Research Program at the Georgia Institute of Technology.

References

[1] Rika Antonova, Jingyun Yang, Priya Sundaresan, Dieter Fox, Fabio Ramos, and Jeannette Bohg. A bayesian treatment of real-to-sim for deformable object manipulation. *IEEE Robotics and Automation Letters*, 7(3):5819–5826, 2022. 3

[2] Armen Avetisyan, Manuel Dahnert, Angela Dai, Manolis Savva, Angel X Chang, and Matthias Nießner. Scan2cad: Learning cad model alignment in rgb-d scans. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, pages 2614–2623, 2019. 3

[3] Apurva Badithela, David Snyder, Lihan Zha, Joseph Mikhail, Matthew O'Kelly, Anushri Dixit, and Anirudha Majumdar. Reliable and scalable robot policy evaluation with imperfect simulators. *arXiv preprint arXiv:2510.04354*, 2025. 2, 3, 21

[4] Jose Barreiros, Andrew Beaulieu, Aditya Bhat, Rick Cory, Eric Cousineau, Hongkai Dai, Ching-Hsin Fang, Kunimatsu Hashimoto, Muhammad Zubair Irshad, Masha Itkina, et al. A careful examination of large behavior models for multitask dexterous manipulation. *arXiv preprint arXiv:2507.05331*, 2025. 2

[5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 2, 7, 21

[6] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. 2, 21

[7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. 21

[8] Sylvain Calinon, Florent D'halluin, Eric L. Sauser, Darwin G. Caldwell, and Aude Billard. Learning and reproduction of gestures by imitation. *IEEE Robotics and Automation Magazine*, 17, 2010. 21

[9] Berk Calli, Arjun Singh, Aaron Walsman, Siddhartha Srinivasa, Pieter Abbeel, and Aaron M. Dollar. The ycb object and model set: Towards common benchmarks for manipulation research. In *2015 International Conference on Advanced Robotics (ICAR)*, pages 510–517, 2015. doi: 10.1109/ICAR.2015.7251504. 47

[10] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2025. 3, 5, 22

[11] Chuhao Chen, Isabella Liu, Xinyue Wei, Hao Su, and Minghua Liu. Freeart3d: Training-free articulated object generation using 3d diffusion. In *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*, pages 1–13, 2025. 3

[12] Zoey Chen, Aaron Walsman, Marius Memmel, Kaichun Mo, Alex Fang, Karthikeya Vemuri, Alan Wu, Dieter Fox, and Abhishek Gupta. Urdformer: A pipeline for constructing articulated simulation environments from real-world images. *arXiv preprint arXiv:2405.11656*, 2024. 3

[13] Shuo Cheng, Liqian Ma, Zhenyang Chen, Ajay Mandlekar, Caelan Garrett, and Danfei Xu. Generalizable domain adaptation for sim-and-real policy co-training. *arXiv preprint arXiv:2509.18631*, 2025. 2, 21

[14] Gunjan Chhablani, Xiaomeng Ye, Muhammad Zubair Irshad, and Zsolt Kira. Embodiedsplat: Personalized real-to-sim-to-real navigation with gaussian splats from a mobile device. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 25431–25441, 2025. 3, 21

[15] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The Int'l Journal of Robotics Research*, 2023. 21

[16] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. http://pybullet.org, 2016–2021. 5

[17] Tianyuan Dai, Josiah Wong, Yunfan Jiang, Chen Wang, Cem Gokmen, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. Automated creation of digital cousins for robust policy learning. *arXiv preprint arXiv:2410.07408*, 2024. 2, 3, 4, 8, 21, 36

[18] Murtaza Dalal, Ajay Mandlekar, Caelan Reed Garrett, Ankur Handa, Ruslan Salakhutdinov, and Dieter Fox. Imitating task and motion planning with visuomotor transformers. In *Conf on Robot Learning*, 2023. 2, 21

[19] Prithwish Dan, Kushal Kedia, Angela Chao, Edward Weiyi Duan, Maximus Adrian Pace, Wei-Chiu Ma, and Sanjiban Choudhury. X-sim: Cross-embodiment learning via real-to-sim-to-real. *arXiv preprint arXiv:2505.07096*, 2025. 2, 3

[20] Frederik Ebert, Yanlai Yang, Karl Schmeckpeper, Bernadette Bucher, Georgios Georgakis, Kostas Daniilidis, Chelsea Finn, and Sergey Levine. Bridge Data: Boosting Generalization of Robotic Skills with Cross-Domain Datasets. In *Robotics: Science and Systems*, 2022. 2, 21

[21] Alejandro Escontrela, Justin Kerr, Arthur Allshire, Jonas Frey, Rocky Duan, Carmelo Sferrazza, and Pieter Abbeel. Gaussgym: An open-source real-to-sim framework for learning locomotion from pixels. *arXiv preprint arXiv:2510.15352*, 2025. 3, 21

[22] Daoyi Gao, Dávid Rozenberszki, Stefan Leutenegger, and Angela Dai. Diffcad: Weakly-supervised probabilistic cad model retrieval and alignment from an rgb image. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024. 3

[23] Caelan Garrett, Ajay Mandlekar, Bowen Wen, and Dieter Fox. Skillmimicgen: Automated demonstration generation for efficient skill learning and deployment. *arXiv preprint arXiv:2410.18907*, 2024. 2, 21

[24] Chenghao Gu, Haolan Kang, Junchao Lin, Jinghe Wang, Duo Wu, Shuzhao Xie, Fanding Huang, Junchen Ge, Ziyang Gong, Letian Li, et al. Igen: Scalable data generation for robot learning from open-world images. *arXiv preprint arXiv:2512.01773*, 2025. 3, 21

[25] Can Gümeli, Angela Dai, and Matthias Nießner. Roca: Robust cad model retrieval and alignment from a single image. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4022–4031, 2022. 3

[26] Siddhant Haldar, Lars Johannsmeier, Lerrel Pinto, Abhishek Gupta, Dieter Fox, Yashraj Narang, and Ajay Mandlekar. Point bridge: 3d representations for cross domain policy learning. *arXiv preprint arXiv:2601.16212*, 2026. 2, 21

[27] Xiaoshen Han, Minghuan Liu, Yilun Chen, Junqiu Yu, Xiaoyang Lyu, Yang Tian, Bolun Wang, Weinan Zhang, and Jiangmiao Pang. Re3sim: Generating high-fidelity simulation data via 3d-photorealistic real-to-sim for robotic manipulation. *arXiv preprint arXiv:2502.08645*, 2025. 2, 3, 21

[28] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. Lrm: Large reconstruction model for single image to 3d. *arXiv preprint arXiv:2311.04400*, 2023. 3

[29] Team Hunyuan3D, Shuhui Yang, Mingxin Yang, Yifei Feng, Xin Huang, Sheng Zhang, Zebin He, Di Luo, Haolin Liu, Yunfei Zhao, Qingxiang Lin, Zeqiang Lai, Xianghui Yang, Huiwen Shi, Zibo Zhao, Bowen Zhang, Hongyu Yan, Lifu Wang, Sicong Liu, Jihong Zhang, Meng Chen, Liang Dong, Yiwen Jia, Yulin Cai, Jiaao Yu, Yixuan Tang, Dongyuan Guo, Junlin Yu, Hao Zhang, Zheng Ye, Peng He, Runzhou Wu, Shida Wei, Chao Zhang, Yonghao Tan, Yifu Sun, Lin Niu, Shirui Huang, Bojian Zheng, Shu Liu, Shilin Chen, Xiang Yuan, Xiaofeng Yang, Kai Liu, Jianchen Zhu, Peng Chen, Tian Liu, Di Wang, Yuhong Liu, Linus, Jie Jiang, Jingwei Huang, and Chunchao Guo. Hunyuan3d 2.1: From images to high-fidelity 3d assets with production-ready pbr material, 2025. URL https://arxiv.org/abs/2506.15442. 22

[30] Auke Jan Ijspeert, Jun Nakanishi, and Stefan Schaal. Movement imitation with nonlinear dynamical systems in humanoid robots. *Proceedings 2002 IEEE Int'l Conf on Robotics and Automation*, 2, 2002. 21

[31] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi 0.5$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 7, 8

[32] Arhan Jain, Mingtong Zhang, Kanav Arora, William Chen, Marcel Torne, Muhammad Zubair Irshad, Sergey Zakharov, Yue Wang, Sergey Levine, Chelsea Finn, et al. Polaris: Scalable real-to-sim evaluations for generalist robot policies. *arXiv preprint arXiv:2512.16881*, 2025. 2, 3, 4, 7, 21, 34, 42, 43

[33] Yash Jangir, Yidi Zhang, Kashu Yamazaki, Chenyu Zhang, Kuan-Hsun Tu, Tsung-Wei Ke, Lei Ke, Yonatan Bisk, and Katerina Fragkiadaki. RobotArena ∞: Scalable robot benchmarking via real-to-sim translation. *arXiv preprint arXiv:2510.23571*, 2025. 2, 3, 21

[34] Guangqi Jiang, Haoran Chang, Ri-Zhao Qiu, Yutong Liang, Mazeyu Ji, Jiyue Zhu, Zhao Dong, Xueyan Zou, and Xiaolong Wang. Gsworld: Closed-loop photo-realistic simulation suite for robotic manipulation. *arXiv preprint arXiv:2510.20813*, 2025. 2, 3, 21

[35] Yunfan Jiang, Ruohan Zhang, Josiah Wong, Chen Wang, Yanjie Ze, Hang Yin, Cem Gokmen, Shuran Song, Jiajun Wu, and Li Fei-Fei. Behavior robot suite: Streamlining real-world whole-body manipulation for everyday household activities. *arXiv preprint arXiv:2503.05652,* 2025. 39, 40

[36] Zhenyu Jiang, Cheng-Chun Hsu, and Yuke Zhu. Ditto: Building digital twins of articulated objects from interaction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5616–5626, 2022. 3

[37] Zhenyu Jiang, Yuqi Xie, Kevin Lin, Zhenjia Xu, Weikang Wan, Ajay Mandlekar, Linxi Fan, and Yuke Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. *arXiv preprint arXiv:2410.24185*, 2024. 2, 4, 21

[38] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, George Drettakis, et al. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 6, 24, 25

[39] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024. 2, 6, 21, 40

[40] Yejin Kim, Wilbert Pumacay, Omar Rayyan, Max Argus, Winson Han, Eli VanderBilt, Jordi Salvador, Abhay Deshpande, Rose Hendrix, Snehal Jauhri, Shuo Liu, Nur Muhammad Mahi Shafiullah, Maya Guru, Ainaz Eftekhar, Karen Farley, Donovan Clay, Jiafei Duan, Arjun Guru, Piper Wolters, Alvaro Herrasti, Ying-Chun Lee, Georgia Chalvatzaki, Yuchen Cui, Ali Farhadi, Dieter Fox, and Ranjay Krishna. Molmospaces: A large-scale open ecosystem for robot navigation and manipulation, 2026. 2, 3

[41] Weicheng Kuo, Anelia Angelova, Tsung-Yi Lin, and Angela Dai. Mask2cad: 3d shape prediction by learning to segment and retrieve. In *European Conference on Computer Vision*, pages 260–277. Springer, 2020. 3

[42] Weicheng Kuo, Anelia Angelova, Tsung-Yi Lin, and Angela Dai. Patch2cad: Patchwise embedding learning for in-the-wild shape retrieval from a single image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12589–12599, 2021. 3

[43] Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. *arXiv preprint arXiv:2410.13882*, 2024. 3, 5, 23

[44] Taeyeop Lee, Bowen Wen, Minjun Kang, Gyuree Kang, In So Kweon, and Kuk-Jin Yoon. Any6d: Model-free 6d pose estimation of novel objects. *CVPR*, 2025. 3

[45] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, et al. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR, 2023. 29

[46] Chengshu Li, Mengdi Xu, Arpit Bahety, Hang Yin, Yunfan Jiang, Huang Huang, Josiah Wong, Sujay Garlanka, Cem Gokmen, Ruohan Zhang, Weiyu Liu, Jiajun Wu, Roberto Martín-Martín, and Li Fei-Fei. Momagen: Generating demonstrations under soft and hard constraints for multi-step bimanual mobile manipulation. In *RSS 2025 Workshop on Whole-body Control and Bimanual Manipulation: Applications in Humanoids and Beyond*, 2025. URL https://openreview.net/forum?id=4ATOUj1k9n. 2

[47] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024. 2, 3, 4, 21, 42

[48] Zizhang Li, Cheng Zhang, Zhengqin Li, Henry Howard-Jenkins, Zhaoyang Lv, Chen Geng, Jiajun Wu, Richard Newcombe, Jakob Engel, and Zhao Dong. Art: Articulated reconstruction transformer. *arXiv preprint arXiv:2512.14671*, 2025. 3

[49] Vincent Lim, Huang Huang, Lawrence Yunliang Chen, Jonathan Wang, Jeffrey Ichnowski, Daniel Seita, Michael Laskey, and Ken Goldberg. Planar robot casting with real2sim2real self-supervised learning. *arXiv preprint arXiv:2111.04814*, 2021. 3

[50] Haotong Lin, Sili Chen, Junhao Liew, Donny Y Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025. 3, 5, 22, 24, 25

[51] Jiayi Liu, Denys Iliash, Angel X Chang, Manolis Savva, and Ali Mahdavi-Amiri. Singapo: Single image controlled generation of articulated parts in objects. *arXiv preprint arXiv:2410.16499*, 2024. 3

[52] Minghua Liu, Mikaela Angelina Uy, Donglai Xiang, Hao Su, Sanja Fidler, Nicholas Sharp, and Jun Gao. Partfield: Learning 3d feature fields for part segmentation and beyond. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9704–9715, 2025. 22

[53] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European conference on computer vision*, pages 38–55. Springer, 2024. 3

[54] Changfeng Ma, Yang Li, Xinhao Yan, Jiachen Xu, Yunhan Yang, Chunshi Wang, Zibo Zhao, Yanwen Guo, Zhuo Chen, and Chunchao Guo. P3-sam: Native 3d part segmentation. *arXiv preprint arXiv:2509.06784*, 2025. 22, 23

[55] Abhiram Maddukuri, Zhenyu Jiang, Lawrence Yunliang Chen, Soroush Nasiriany, Yuqi Xie, Yu Fang, Wenqi Huang, Zu Wang, Zhenjia Xu, Nikita Chernyadev, et al. Sim-and-real co-training: A simple recipe for vision-based robotic manipulation. *arXiv preprint arXiv:2503.24361*, 2025. 2, 4, 21

[56] Ajay Mandlekar, Yuke Zhu, Animesh Garg, Jonathan Booher, Max Spero, Albert Tung, Julian Gao, John Emmons, Anchit Gupta, Emre Orbay, et al. Roboturk: A crowdsourcing platform for robotic skill learning through imitation. In *Conf on Robot Learning*, 2018. 21

[57] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021. 21

[58] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. *arXiv preprint arXiv:2310.17596*, 2023. 2, 21, 29, 39

[59] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Munoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. 5, 29

[60] Saman Motamed, William Harvey, Benjamin Klein, Luc Van Gool, Zhuoning Yuan, and Ta-Ying Cheng. Void: Video object and interaction deletion. *arXiv preprint arXiv:2604.02296*, 2026. 23

[61] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 2, 7, 21, 39

[62] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE Int'l Conf on Robotics and Automation (ICRA)*, 2024. 21

[63] Nicholas Pfaff, Thomas Cohn, Sergey Zakharov, Rick Cory, and Russ Tedrake. Scenesmith: Agentic generation of simulation-ready indoor scenes, 2026. 2, 3

[64] Dean A Pomerleau. Alvinn: An autonomous land vehicle in a neural network. In *Advances in neural information processing systems*, 1989. 21

[65] Xiaowen Qiu, Jincheng Yang, Yian Wang, Zhehuan Chen, Yufei Wang, Tsun-Hsuan Wang, Zhou Xian, and Chuang Gan. Articulate anymesh: Open-vocabulary 3d articulated objects modeling. *arXiv preprint arXiv:2502.02590*, 2025. 5, 23

[66] M Nomaan Qureshi, Sparsh Garg, Francisco Yandun, David Held, George Kantor, and Abhisesh Silwal. Splatsim: Zero-shot sim2real transfer of rgb manipulation policies using gaussian splatting. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6502–6509. IEEE, 2025. 3

[67] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 3

[68] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. In *International Conference on Learning Representations*, volume 2025, pages 28085–28128, 2025. 23

[69] I2RT Robotics. Yam robot arm, 2025. URL https://i2rt.com/collections/yam-arm. 6, 40

[70] Stefan Schaal. Is imitation learning the route to humanoid robots? *Trends in cognitive sciences*, 3, 1999. 21

[71] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-Motion Revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 25

[72] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, et al. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH 2023 conference proceedings*, pages 1–12, 2023. 24, 25

[73] George Tang, William Zhao, Logan Ford, David Benhaim, and Paul Zhang. Segment any mesh. *arXiv preprint arXiv:2408.13679*, 2024. 22, 23

[74] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jiawei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. Sam 3d: 3dfy anything in images, 2025. URL https://arxiv.org/abs/2511.16624. 47

[75] Tencent Hunyuan3D Team. Hunyuan3d 2.5: Towards high-fidelity 3d assets generation with ultimate details, 2025. URL https://arxiv.org/abs/2506.16504. 3

[76] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, Xu Shi, Ning Gao, Hangxu Liu, Xuekun Jiang, Zherui Qiu, Feng Yuan, et al. Interndata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy. *arXiv preprint arXiv:2511.16651*, 2025. 21

[77] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. Triposr: Fast 3d object reconstruction from a single image. *arXiv preprint arXiv:2403.02151*, 2024. 3

[78] Marcel Torne, Arhan Jain, Jiayi Yuan, Vidaaranya Macha, Lars Ankile, Anthony Simeonov, Pulkit Agrawal, and Abhishek Gupta. Robot learning with super-linear scaling. *arXiv preprint arXiv:2412.01770*, 2024. 3, 21

[79] Marcel Torne, Anthony Simeonov, Zechu Li, April Chan, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. *arXiv preprint arXiv:2403.03949*, 2024. 2, 3, 21

[80] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Trans. Pattern Anal. Mach. Intell.*, 13(4):376–380, April 1991. ISSN 0162-8828. doi: 10.1109/34.88573. URL https://doi.org/10.1109/34.88573. 24

[81] Xiaogang Wang, Bin Zhou, Yahao Shi, Xiaowu Chen, Qinping Zhao, and Kai Xu. Shape2motion: Joint analysis of motion parts and attributes from 3d shapes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8876–8884, 2019. 3

[82] Xinjie Wang, Liu Liu, Yu Cao, Ruiqi Wu, Wenkang Qin, Dehui Wang, Wei Sui, and Zhizhong Su. Embodiedgen: Towards a generative 3d world engine for embodied intelligence. *arXiv preprint arXiv:2506.10600*, 2025. 3

[83] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. In *Forty-first Int'l Conf on Machine Learning*, 2023. 21

[84] Zehan Wang, Siyu Chen, Lihe Yang, Jialei Wang, Ziang Zhang, Hengshuang Zhao, and Zhou Zhao. Depth anything with any prior, 2025. URL https://arxiv.org/abs/2505.10565. 22

[85] Ziqian Wang, Yonghao He, Licheng Yang, Wei Zou, Hongxuan Ma, Liu Liu, Wei Sui, Yuxin Guo, and Hu Su. Tabletopgen: Instance-level interactive 3d tabletop scene generation from text or single image, 2025. 2, 3

[86] Adam Wei, Abhinav Agarwal, Boyuan Chen, Rohan Bosworth, Nicholas Pfaff, and Russ Tedrake. Empirical analysis of sim-and-real cotraining of diffusion policies for planar pushing from pixels. *arXiv preprint arXiv:2503.22634*, 2025. 2, 21

[87] Xinyue Wei, Minghua Liu, Zhan Ling, and Hao Su. Approximate convex decomposition for 3d meshes with collision-aware concavity and tree search, 2022. 5, 23

[88] Bowen Wen, Jonathan Tremblay, Valts Blukis, Stephen Tyree, Thomas Müller, Alex Evans, Dieter Fox, Jan Kautz, and Stan Birchfield. Bundlesdf: Neural 6-dof tracking and 3d reconstruction of unknown objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 606–617, 2023. 3

[89] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. Foundationpose: Unified 6d pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 3, 5, 22, 47

[90] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. Foundationstereo: Zero-shot stereo matching. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5249–5260, 2025. 3, 5, 22

[91] Bowen Wen, Shaurya Dewan, and Stan Birchfield. Fast-FoundationStereo: Real-time zero-shot stereo matching. *CVPR*, 2026. 3

[92] Yijia Weng, Bowen Wen, Jonathan Tremblay, Valts Blukis, Dieter Fox, Leonidas Guibas, and Stan Birchfield. Neural implicit representation for building digital twins of unknown articulated objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3141–3150, 2024. 3

[93] Kailu Wu, Fangfu Liu, Zhihan Cai, Runjie Yan, Hanyang Wang, Yating Hu, Yueqi Duan, and Kaisheng Ma. Unique3d: High-quality and efficient 3d mesh generation from a single image. *Advances in Neural Information Processing Systems*, 37:125116–125141, 2024. 3

[94] Hongchi Xia, Chih-Hao Lin, Hao-Yu Hsu, Quentin Leboutet, Katelyn Gao, Michael Paulitsch, Benjamin Ummenhofer, and Shenlong Wang. Holoscene: Simulation-ready interactive 3d worlds from a single video, 2025. 3

[95] Hongchi Xia, Entong Su, Marius Memmel, Arhan Jain, Raymond Yu, Numfor Mbiziwo-Tiapo, Ali Farhadi, Abhishek Gupta, Shenlong Wang, and Wei-Chiu Ma. Drawer: Digital reconstruction and articulation with environment realism. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21771–21782, 2025. 3, 21

[96] Hongchi Xia, Xuan Li, Zhaoshuo Li, Qianli Ma, Jiashu Xu, Ming-Yu Liu, Yin Cui, Tsung-Yi Lin, Wei-Chiu Ma, Shenlong Wang, Shuran Song, and Fangyin Wei. Sage: Scalable agentic 3d scene generation for embodied ai, 2026. 2, 3

[97] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3d generation. *Tech report*, 2025. 3, 22, 23

[98] Jiale Xu, Weihao Cheng, Yiming Gao, Xintao Wang, Shenghua Gao, and Ying Shan. Instantmesh: Efficient 3d mesh generation from a single image with sparse-view large reconstruction models. *arXiv preprint arXiv:2404.07191*, 2024. 3

[99] Zihao Yan, Ruizhen Hu, Xingguang Yan, Luanmin Chen, Oliver Van Kaick, Hao Zhang, and Hui Huang. Rpm-net: recurrent prediction of motion and parts from point cloud. *arXiv preprint arXiv:2006.14865*, 2020. 3

[100] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 7, 39

[101] Chenghao Yin, Da Huang, Di Yang, Jichao Wang, Nanshu Zhao, Chen Xu, Wenjun Sun, Linjie Hou, Zhijun Li, Junhui Wu, et al. Genie sim 3.0: A high-fidelity comprehensive simulation platform for humanoid robot. *arXiv preprint arXiv:2601.02078*, 2026. 2, 3, 21

[102] Justin Yu, Letian Fu, Huang Huang, Karim El-Refai, Rares Andrei Ambrus, Richard Cheng, Muhammad Zubair Irshad, and Ken Goldberg. Real2render2real: Scaling robot data without dynamics simulation or robot hardware. *arXiv preprint arXiv:2505.09601*, 2025. 3, 21

[103] Sylvia Yuan, Ruoxi Shi, Xinyue Wei, Xiaoshuai Zhang, Hao Su, and Minghua Liu. Larm: A large articulated object reconstruction model. In *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*, pages 1–12, 2025. 3

[104] Kaifeng Zhang, Shuo Sha, Hanxiao Jiang, Matthew Loper, Hyunjong Song, Guangyan Cai, Zhuo Xu, Xiaochen Hu, Changxi Zheng, and Yunzhu Li. Real-to-sim robot policy evaluation with gaussian splatting simulation of soft-body interactions. *arXiv preprint arXiv:2511.04665*, 2025. 2, 3, 21

[105] Siheng Zhao, Jiageng Mao, Wei Chow, Zeyu Shangguan, Tianheng Shi, Rong Xue, Yuxi Zheng, Yijia Weng, Yang You, Daniel Seita, et al. Robot learning from any images. In *Conference on Robot Learning*, pages 4226–4245. PMLR, 2025. 2, 3, 21

[106] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. 21

Appendix

A. Overview

The Appendix contains the following content.

- **FAQ** (Appendix B): answers to some common questions
- **Limitations** (Appendix C): more thorough list and discussion of SimFoundry limitations
- **Full Related Work** (Appendix D): more thorough discussion on related work
- **Scene Reconstruction** (Appendix E): additional details on SimFoundry scene reconstruction method
- **Digital Cousins Augmentation** (Appendix F): additional details on how SimFoundry produces cousins at the object, scene and task level
- **Detailed Experiment Results** (Appendix G): tables containing detailed results for our real-to-sim and sim-to-real evaluations as well as additional discussion on these results
- **Policy Details** (Appendix H): details on how policies are obtained for all SimFoundry experiments
- **Robot Platform and Task Details** (Appendix I): details on the robot platforms, how they are modeled in simulation, and the tasks used in the experiments
- **Real-to-Sim Policy Evaluations** (Appendix J): additional details on real-to-sim policy evaluation experiments
- **Human Interaction** (Appendix K): details on how users can interact with SimFoundry to improve generation quality or control specific elements
- **System Analysis** (Appendix L): analysis of several characteristics of SimFoundry, including reconstruction fidelity and environment generation throughput. This section also details the background reconstruction methods supported by SimFoundry

B. FAQ

1. **Why should I use SimFoundry compared to alternative methods that generate simulation environments?**
   There have been many impressive systems showcasing automated, diverse, and scalable real-to-sim reconstruction pipelines. We highlight the following key distinguishing details that are especially advantageous:
   - **Towards Feature-Complete Automation.** Our approach is *fully* automated, supporting programmatic scene, articulated object, and background reconstruction, all within a single unified pipeline.
   - **Empirically Validated for Robotics Tasks.** We show that SimFoundry is concretely useful for downstream robotics applications, both in real-to-sim eval and sim-to-real training settings.

   See Appendix D for more discussion.
2. **How are SimFoundry scenes tested for physical stability?**
   Our reconstructed scenes are spawned within a PyBullet physics simulator instance, and subsequently stepped until objects settle. This guarantees physical stability during subsequent initializations, though objects may drift with respect to their original fitted poses. See Appendix E.4 for more details.
3. **How much manual effort is needed to produce SimFoundry scenes and use them for the robot manipulation applications shown in the paper?**
   After initial manual scene scans using a smartphone camera, our SimFoundry environments used in our robotics experiments were generated using our fully automated pipeline and then quickly interactively tuned with a human operator, requiring a few minutes worth of iteration.
4. **What are some typical runtimes for how long it takes to generate a scene?**
   On average, it takes roughly 5 minutes per object when reconstructing a real scene in simulation, which is the time cost amortized across the entire pipeline.
5. **Does SimFoundry support transparent objects?**
   Yes, with a caveat. SimFoundry merely inherits the strengths and limitations of the underlying 2D to 3D mesh model used to generate simulation meshes. The default model used is Hunyuan 2.1, which does not support transparency, though we additionally support other models such as TRELLIS.2, which does support transparent objects.
6. **For the DROID experiments, how much of the policy performance can be attributed to the pretrained checkpoint?** We find that for the tasks showing sim-to-real transfer on DROID, the $\pi_{0.5}$ and π_0 checkpoints pre-trained on the DROID dataset perform poorly without task-specific finetuning data. On **Store Marker** and **Throw Away Trash**, both checkpoints get a 0% success rate, while for **Stack Dishware**, π_0-DROID gets 52% and $\pi_{0.5}$-DROID gets 48% success rate (improving to 100% with sim-only finetuning). This illustrates that SimFoundry data is valuable for finetuning pretrained checkpoints, especially on difficult tasks.
7. **Why do you use a binary success metric instead of a normalized task reward?** We primarily report end-to-end task success because it provides a stricter test of real-to-sim fidelity. Normalized rewards can give partial credit for completing intermediate subtasks, whereas binary success requires the policy to complete the full task under the reconstructed scene dynamics, visuals, and geometry. Empirically, we find that success-rate correlations are harder to achieve in this setting, especially for long-horizon tasks and real-world-finetuned policies, where small reconstruction errors can cause policies to go out-of-distribution and fail. Therefore, maintaining a strong success-rate correlation in this setting provides stronger evidence that SimFoundry reconstructions faithfully preserve the factors that determine real-world policy performance.

C. Limitations

As all VLMs in this paper are queried through a remote third-party provider, identical inputs can yield non-deterministic outputs across runs; for instance, we occasionally observe inconsistent inpainting from the Gemini image model, producing degenerate or duplicated extracted objects.

Reconstruction fidelity is further bounded by the quality of the inferred point cloud. For monocular inputs in particular, the scale and shape of the recovered geometry may not fully match the real-world scene, reducing the accuracy of the reconstructed output. Our articulation results likewise depend on accurate 3D segmentation of the object mesh, which can be difficult for meshes produced by image-to-mesh models or for objects with occluded internal structure.

Our physics-stability procedure assumes that objects rest on a single flat reference surface, which restricts the pipeline to tabletop-style scene layouts. Future work could address this to extend to more complex and varied scenes.

Finally, the automatic background pipeline removes the second-capture and manual-alignment work, but at the cost of runtime: the two-pass video inpainting required to produce a dense, temporally consistent, and clean RGB-D stream for splat training takes roughly 90 minutes per scene on a single GPU. This overhead is largely hidden in a multi-GPU setting, where background reconstruction can run in parallel with the Extraction, Generation, and Variation stages rather than serially after them. Also, we use mesh reconstructions of the background for some of our robotics experiments in simulation, due to issues with near-field clipping of the generated 3DGS and to reduce rendering latency.

D. Full Related Work

D.1. Real–to–Sim for Simulation Environment Creation and Applications

We expand on the main-text discussion and position SimFoundry against four categories of prior automated real-to-sim work.

Real–to–sim–to–real manipulation. This category digitizes physical scenes into simulation for real–world robot training [17, 24, 78, 79, 95, 105]. These systems are typically demonstrated on single-step pick-and-place tasks with rigid objects, whereas SimFoundry supports a wider manipulation regime that includes bimanual setups, articulated-object interactions, and multi-step tasks. A further distinction is that most prior pipelines produce a static digital twin of the captured scene, which bounds environmental diversity and limits the generalization gains achievable from simulated training. SimFoundry instead generates multiple *cousins* of each reconstructed scene—object, scene, and task cousins—which prove to be crucial to improve sim-to-real policy transfer, extending the digital-cousins formulation of [17].

Real–to–sim–to–real for navigation and locomotion. Related approaches close the same loop for robot navigation [14] and locomotion [21], but focus on tasks with substantially different physics and contact requirements from dexterous manipulation.

Real–to–sim policy evaluation. Another body of work focuses on using reconstructed simulation environments as a way to reliably evaluate manipulation policies [3, 27, 32–34, 47, 101, 104], such that the results strongly correlate with corresponding real-world evaluations. However, unlike SimFoundry, several such systems do not show that data from their simulation environments can be used to train real–world agents [3, 32, 33, 47, 104], and to our knowledge, no systems support tasks involving general articulated objects.

Real–to–render. A separate line circumvents physical simulation entirely and uses the reconstructed scene purely as a rendering target [102]. This bypasses the cost of physics modeling but, by the same token, makes it difficult to apply to higher-precision contact-rich tasks that depend on accurate dynamics.

SimFoundry belongs to a select group of real–to–sim systems [27, 34, 101] that demonstrate both successful sim-to-real agent transfer and strong correlation between simulated and physical policy evaluations. However, SimFoundry goes beyond these systems by handling more diverse task conditions (including bimanual, articulation, and multi–step manipulation) and supporting multiple types of cousins of a reconstructed sim-ready scene to fully unlock the diversity of the simulation (see Table 1 for a summary).

D.2. Imitation Learning from Human Demonstrations and Synthetic Data Generation.

Robot teleoperation [56, 106] is a common approach for collecting demonstrations to train robots to perform manipulation tasks autonomously – here, a human uses a teleoperation device (such as a smartphone or a VR controller) to guide a robot through different tasks, and the resultant robot sensor streams and actions are logged to a dataset. Robot manipulation policies are often trained on such datasets with Behavioral Cloning (BC) [15, 30, 57, 64, 70]. In recent years, this approach has been scaled up to collect months of data using large teams of human operators [6, 20, 39, 62], and has proven to be very effective for robot manipulation [5, 7, 8, 39]. However, data collection is a bottleneck, since it is time–consuming and expensive. A recent line of work leverages synthetic data generation (SDG) in simulation [18, 23, 37, 58, 83] as a compelling alternative to address the need for large-scale datasets. Recent evidence has shown that these synthetic datasets can supplement or even replace real-world datasets to reduce the burden of real-world data collection [13, 26, 55, 61, 76, 86, 101]. We use such tools to highlight an important application of our system – real–to–sim–to–real policy learning. Here, we reconstruct a simulation environment (along with controlled variations) from a real-world environment, generate synthetic data in simulation, and train manipulation agents that transfer to the real-world, all with minimal human effort.

E. Scene Reconstruction

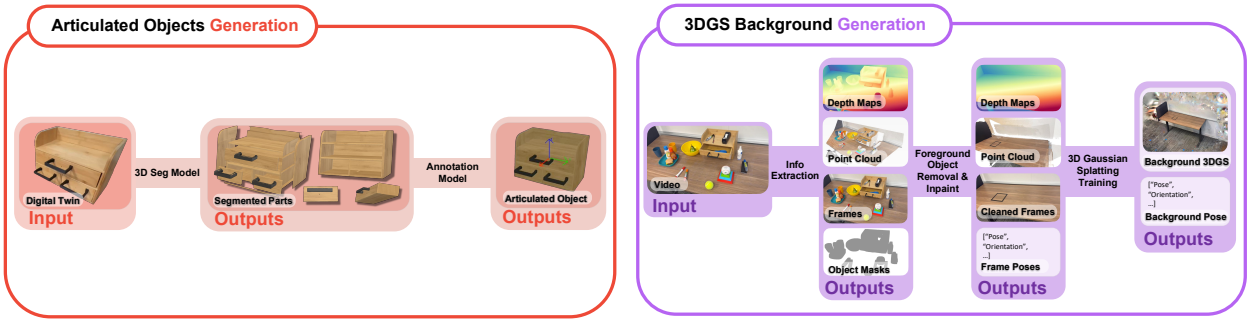


Figure E.1: **Articulation and 3DGS Background Pipeline Overview.** SimFoundry generates articulated objects by first decomposing a pre-existing mesh into subsequent parts, which are then annotated with relevant joint types, locations, and ranges via a VLM. SimFoundry also can automatically generate a high-fidelity 3DGS background by first generating a synthetic video with removed foreground, extracting extrinsics, and training a 3DDGS to reconstruct the scene geometry.

E.1. Extraction Details

The extraction stage converts a raw video into a representative RGB-D frame and converts this frame into a scene point cloud. Then, these modalities are used as inputs to iteratively segment foreground objects, removing detected objects with image and depth inpainting such that the subsequent foreground object can be detected from the residual scene. This process produces the per-object RGB-D crops and masks used by the mesh-generation and pose-alignment stages.

Representative Frame Selection For video inputs, our current pipeline uses frame 0 as the default representative frame to reconstruct scenes. We ask users to start recording videos from a clear point of view that capture the whole scene; ideally from a front-facing view that minimizes occlusion.

E.2. Foundation Model Details

SimFoundry is intended to be modular, and supports multiple foundation models that can be changed during execution. Below, we show the models our pipeline currently supports for each type of foundation model V_*:

- $V_{im2depth}$: If the input is a single image or video, we utilize DepthAnything3 [50]; if the input is a stereo image pair, we utilize FoundationStereo [90].
- V_{seg}^{image} : We utilize SAM3 [10].
- V_{scene} : We utilize Gemini-Pro-3, though any Gemini or other general purpose VLM can be used by our pipeline.
- V_{image} : We utilize Gemini-Pro-3-Image-Preview, though any Gemini or other general purpose image-editing VLM can be used by our pipeline.
- $V_{inpaint}^{depth}$: We utilize PriorDepthAnything [84].
- V_{mesh}: We utilize either Hunyuan2.1 [29] or TRELLIS.2 [97].
- V_{pose}: We utilize FoundationPose [89] to refine the 6D pose of the generated mesh with respect to the depth map.
- $V_{articulation}$: We utilize Gemini-Pro-3, though any Gemini or other general purpose VLM can be used by our pipeline.
- V_{seg}^{mesh} : We utilize mainly P3-SAM [54], although our pipeline also supports Segment Any Mesh [73] and Partfield [52].

- V_{seg}^{video}: We utilize SAM2 [68] to propagate the keyframe foreground mask produced by V_{seg}^{image} through the remainder of the video.
- $V_{inpaint}^{video}$: We utilize VOID [60] as a two-pass chunked inpainting model to remove foreground pixels from the masked frames and synthesize a clean static-scene RGB stream.

E.3. Articulated Object Generation

In this section we detail our articulated object generation pipeline, which extends prior methods such as Articulate Anymesh [65] and Articulate Anything [43].

Segmentation We first render views of the object from multiple angles, pass these into a VLM $V_{articulation}$, and prompt $V_{articulation}$ to list the different parts of the object that can be articulated and the types of joints. For example, this would be a drawer (prismatic) for a cabinet or a door (revolute) for a microwave. We then segment the mesh of the object with a mesh segmentation model V_{seg}^{mesh} , such as P3-SAM [54] or Segment Any Mesh [73] which assigns an integer label to every face; we refer to each maximal group of faces sharing a label as a segment. Most existing methods only assign labels to external surfaces, but since meshes generated by TRELLIS.2 [97] can have internal structures, we propagate these labels to unlabeled mesh faces using a majority-vote label propagation over the face-adjacency. Mesh segmentation typically yields many more segments than there are articulatable parts (the semantic components named by $V_{articulation}$ in the previous step), i.e. the model is over-segmented — a single drawer may span several segments. To recover the parts, we render the object again from multiple views with each segment shown in a distinct color, and prompt $V_{articulation}$ to assign every segment to one of the previously identified parts. Segments mapped to the same part are then merged into a single mesh, yielding one mesh per articulatable part. This segmentation and assignment can optionally be refined by the user via a GUI.

Joint Generation We adapt the actor-critic algorithm and API from Articulate Anything [43] to generate the joint parameters. We prompt $V_{articulation}$ to predict the joint axes and placements of each part by providing a python API which can generate URDFs. The API allows $V_{articulation}$ to place joints relative to parts (for example, a revolute joint can be placed along the left edge of a door), which helps ground $V_{articulation}$ and simplify its task. $V_{articulation}$ generates code which calls this API, and the result is compiled into a URDF. We then move the joints of the object according to this URDF in a simulator, and render a video of this movement. The video is then judged by a separate critic VLM, which is asked to rate the accuracy and realism of the movement and provide feedback for improvement if necessary. $V_{articulation}$ is prompted to improve its prediction by incorporating this feedback, and this process continues until the critic gives a score above a threshold.

Physical Parameters Finally, we prompt $V_{articulation}$ to generate physical parameters such as link mass, joint friction, and damping. We provide $V_{articulation}$ with the calculated volume of each part and the entire object to better contextualize its predictions.

E.4. Object Depenetration and Physical Stability

Although objects' poses are estimated by foundation models, minor estimation errors can still lead to interpenetration between neighboring objects. To generate a physically plausible scene, we perform a depenetration step.

First, we generate objects' respective collision meshes using CoACD [87]. Then, the reconstructed scene is spawned in PyBullet, after which the physics simulation is subsequently stepped (and force-setting objects' velocities to be zero after every step to avoid potential explosions from de-penetration) until the object poses settle. This final set of poses is then cached, guaranteeing that the scene will be physically stable during subsequent initializations.

E.5. Background Reconstruction and Alignment

We provide two routes for reconstructing the static background as a 3D Gaussian Splat [38] and registering it with the SimFoundry reconstructed scene's world frame: (a) a fully automatic pipeline that reuses the single raw capture from Extraction, and (b) a manual pipeline that leverages a second foreground-free capture. The two emit identical asset structures and differ only in how the splat is obtained and aligned. We describe each in turn and conclude with a comparison of their trade-offs.

E.5.1. Automatic Background Reconstruction and Alignment

The automatic pipeline runs end-to-end from the same raw capture used by Extraction and emits a splat rigidly aligned to the reconstructed scene's world frame. It comprises four phases: (1) frame preparation and foreground inpainting, (2) metric depth and pose recovery, (3) depth-supervised splat training, and (4) the rigid bridge into the simulator world frame.

Frame preparation and foreground inpainting. We uniformly subsample the source video to a fixed frame budget that balances dense viewpoint coverage against downstream memory. To construct a per-frame foreground mask for all foreground object, we first query V_{scene} on a single keyframe to enumerate the foreground object categories, refine those prompts into pixel-accurate masks with V_{seg}^{image}, and propagate the merged keyframe mask temporally with a video segmentation model V_{seg}^{video}. The resulting binary mask stack drives a two-pass video inpainting model $V_{inpaint}^{video}$: the first pass fills masked regions with a plausible static-scene completion, and the second re-inpaints residual hallucinations using the first pass as conditioning. We run $V_{inpaint}^{video}$ in chunks to respect GPU memory, yielding a sequence of clean RGB frames with no foreground objects.

Metric depth and pose recovery. We apply DepthAnything3 [50] to two frame streams: (a) the original frames, to recover sharp camera poses unbiased by inpainting artifacts, and (b) the inpainted frames, to recover depth consistent with the RGB stream the splat trains against. Since DepthAnything3 [50]'s forward pass is limited to a bounded number of frames, we process it in overlapping chunks; within each chunk it returns metric depth maps, per-pixel confidences, intrinsics, and world-to-camera extrinsics in a local reference frame. We merge chunks into a single trajectory by fitting an Umeyama [80] similarity between each chunk and the first on the shared camera centers in their overlap region; this recovers scale and translation to within millimeter residuals under smooth capture motion.

The inpainted-stream and original-stream depth maps live in two independent metric worlds due to two independent DepthAnything3 [50] forward passes, and may not align well with each other. Hence, we back-project the inpainted depth into a dense point cloud, downsample it, and apply a second Umeyama fit to align it into the original-stream world. The transformed cloud is exported as a seed PLY that initializes the splat's positions, so training begins from a geometrically plausible cloud rather than a random prior.

Depth-supervised splat training. We train a 3D Gaussian Splat in NerfStudio [72] against the inpainted RGB frames, using the *original-stream* camera poses and the seed PLY as initialization, under two training losses: a standard photometric loss between rendered and ground-truth inpainted RGB, and an $L1$ depth loss between rendered depth and the inpainted-stream depth from DepthAnything3 [50], masked by a per-pixel confidence threshold and weighted by a fixed coefficient. Pairing photometric and depth supervision on the *same* inpainted RGB-D stream keeps the two losses mutually consistent; the depth term suppresses floaters that purely photometric training tends to place above textureless flat surfaces to explain view-dependent shading.

To absorb residual misalignment between the original-stream poses (used for camera placement) and the inpainted-stream depth (used for geometric supervision), which arises from DepthAnything3 [50] sub-pixel pose noise and the small pose offset induced by inpainting, we enable a per-camera $\mathrm{SO}(3)\times\mathbb{R}^3$ pose optimizer that learns a small rigid perturbation per training camera. We find this to be the single most impactful design choice for splat sharpness: without it, the splat is consistently blurry regardless of frame count, resolution, or iteration budget.

Rigid bridge into the simulator world. The trained splat lives in the original-stream camera world. To register it with the reconstructed scene—expressed in a ground-plane-aligned world frame estimated by Extraction—we compose two transforms: the cam2world pose of an anchor frame from DepthAnything3 [50] on the original stream, and the cam2world of the same anchor frame from Extraction's ground-plane fit. Their composition is a single rigid transform $M_{src \to og}$ mapping any point in the splat's world into the simulator world. The registered splat is added alongside the per-object meshes from Generation as an additional scene asset, producing a simulation-ready scene whose static geometry and appearance faithfully reproduce the captured environment.

E.5.2. Manual Background Alignment

When the user has physical access to the captured environment, a more stable alternative is available: record a *second* video of the same scene with all foreground objects removed, train the splat directly on this clean stream, and align the result through the SimFoundry interactive scene editor. We describe this path in three phases.

Clean-stream capture and pose estimation. The user re-films the same environment with the same camera and a similar trajectory after physically clearing the foreground objects. We process the video through the standard Nerfstudio [72] pipeline, which uniformly extracts frames and recovers camera intrinsics and per-frame extrinsics via COLMAP [71] structure-from-motion. The pipeline emits the successfully registered frames together with a sparse SfM point cloud initializing the splat.

Splat training on the foreground-free video. We train a `splatfacto-big` 3D Gaussian Splat [38] on the registered frames via Nerfstudio's `ns-train` entrypoint, supervised only by the standard photometric loss against the captured RGB frames, and export it to a PLY via `ns-export` for downstream loading. The absence of inpainted RGB content typically yields a noticeably sharper reconstruction than the automatic pipeline, particularly on textureless flat surfaces and along silhouettes where the automatic pipeline must synthesize plausible content.

Interactive alignment via the scene editor. Unlike the automatic pipeline—where the splat's world frame is linked to the foreground capture through shared poses and can therefore be bridged into the simulator world by a derived rigid transform—the clean-stream capture shares no common camera trajectory with the original capture, and the COLMAP world it lives in is metrically arbitrary up to a similarity. There is thus no obvious automated way to compute its alignment with the reconstructed scene. Instead, we expose the trained splat as an interactive prim inside the SimFoundry scene editor. The user is presented with a side-by-side rendering of the foreground scene (per-object meshes from Generation in their estimated poses) and the background splat, and applies $\mathrm{SE}(3)$ transformations (3-DoF translation, 3-DoF rotation) and an isotropic scale to the splat prim via keyboard commands. Once satisfied, the final $M_{src \to og}$ transform is serialized alongside the splat asset so subsequent launches load the manually-aligned scene without further intervention.

E.5.3. Comparison and Trade-offs

The two pipelines emit identical asset structures and achieve comparable reconstruction quality, differing only in capture requirements and the failure modes inherited from their respective reconstruction paths.

The **automatic pipeline** requires only the single video already captured for Extraction and no user interaction, making it the preferred choice when capture effort must be minimized, when only one video is available, or when the scene is impractical to clear—for example, scenes with fixed installations, public spaces, or archival footage. Because its background is bridged analytically into the original capture's camera frame, it inherently reproduces the exact viewpoint geometry, making it the preferred choice when high reproducibility from a specific camera frame is required (reflected in its consistently higher alignment scores in Table L.6). Its principal limitation is that the inpainting model must synthesize content behind removed foreground objects, which can introduce subtle hallucinations on textureless flat surfaces and along object silhouettes. Additionally, the

foreground inpainting step for two-pass denoising requires around 90 minutes on a single NVIDIA RTX 3090. However, this process is independent of the main SimFoundry pipeline and can run in parallel in a multi-GPU setting.

The **manual pipeline** trades a second capture for higher background surface fidelity: because it trains directly on a genuinely foreground-free video, it avoids inpainting artifacts entirely and produces sharper reconstructions on exactly the surfaces where the automatic pipeline struggles. It is preferable when reconstruction quality is the dominant concern and a second capture is feasible. Its costs are the additional capture effort, physical access to clear the scene, and a brief manual alignment step, since the clean stream shares no camera trajectory with the original capture and cannot be bridged automatically; consequently, its background cannot be registered to the original viewpoint as precisely as the automatic pipeline's, and exact alignment is difficult to achieve by hand.

For qualitative and quantitative comparison between two pipelines, refer to Appendix L.2.

F. Digital Cousins Augmentation

F.1. Object Cousins Augmentation

To systematically generate diverse yet realistic object cousins, we make use of an automated generation pipeline using a VLM and an image generation model. This approach uses the segmented object appearance to create object cousins that maintain the identity of the original object and extend its distribution in terms of shape, structure, and appearance.

The pipeline operates through the following steps:

1. **Object Canonicalization and Context Extraction**: For a given reconstructed object, the input of this system includes the isolated object image with a transparent background, which is generated by the reconstruction pipeline. In addition, the original scene image is also retrieved, allowing generated object variants to be compatible with their surrounding environment. The object name is also canonicalized by removing irrelevant information, including material, size, and transient states.
2. **Functional Component Decomposition**: First, the VLM is prompted to decompose the object into functional components based on grasp affordance. This process will result in a structured list of parts, such as handle, lid, body, base, etc., and this will provide a semantically meaningful basis for localized cousin generation rather than applying unconstrained variation to the object as a whole.
3. **Dimension-Specific Cousin Proposal**: For each functional component, the VLM proposes multiple candidate cousins along three predefined dimensions: *geometry*, *topology*, and *visual appearance*. Geometry changes involve continuous shape attributes, topology changes involve the structure of the object, and visual variations involve surface-level properties such as texture or material appearance. To ensure that the model is realistic, it is specifically instructed to only create deterministic, everyday object variants and to avoid implausible or unusual modifications. To this end, the model is also not allowed to create unrealistic cousins by altering the topology of the object if it is not feasible.
4. **Scene-Aware Image Synthesis**: Each of the proposed component-level variation is then applied by the image generation model to generate a modified image of the object. The model is asked to change only the specified component, keep other components unchanged, maintain a realistic appearance, and produce the result with a semi-transparent background. The scene image is also given as input for better matching of the generated object image with the original scene image.
5. **Reasonableness Verification and Structured Output**: To filter out low-quality generations, each synthesized object is checked by the VLM for plausibility in the real world and scene consistency. Those found to be implausible or inconsistent with the scene are discarded, with a bounded fallback approach maintained for coverage over variation dimensions. The accepted results are then stored, along with relevant metadata information such as the identity of the components, variation dimension, textual descriptions, prompts, and verification results, to make them readily usable by subsequent data generation and simulation pipelines.

The detailed prompt template used for object cousins augmentation is shown in Fig F.1.

F.2. Scene Cousins Augmentation

Starting from the canonical spatial arrangement of objects in the generated scene, we apply randomization to vary their relative placements *semantically*. For instance, if the reconstructed scene has a spoon placed to the right of a plate, we vary the spoon's placement to be placed on top of, or to the left of, the plate. Additionally, we select and place distractor objects that are feasible for a given scene. This augmentation has the following steps:

1. **Predicate Sampling**: for a given scene and task, an anchor object is selected. This anchor object can be selected automatically by a VLM based on the task, or by the user. Then, for each of the other objects, a spatial predicate (or multiple predicates) is selected from the following list: [**LeftOf, RightOf, InFrontOf,**

VLM Prompts for Object Cousins Generation

```
Prompt A: Functional Component Decomposition

With the given object image, decompose the object into functional components based on grasp affordance, list the component
      name only in number list, if there is a main body, treat its edge or bottom as a same component inside main body. Make
      sure to only generate the list of component names, don't generate any other additional text.

Prompt B: Topology Feasibility Check

For the {component} of the object shown, is it reasonable to make variation on topology such that the modified object still
      looks realistic and is reasonable enough that we would see this object in our daily task? Answer with simply yes or no.
       If it is likely not to be normal, make it a no.

Prompt C: Component-wise Variation Proposal

For the component '{component}', list {num_geometry} ways to generate variation through Geometry dimension, {num_topology}
      ways through Topological dimension, and {num_visual} ways through Visual dimension:

1. Geometry dimension
2. Topological dimension
3. Visual dimension

Make sure to be determined in the description, prevent using words like or, such as, like, or similar to in the description.
Make sure to still maintain the object to have a realistic and reasonable look like something you would see in the daily
      life.
For example, a red banana and a banana with a hole in it is a bad variation, because we rarely see those in daily life.
But a banana with green skin is a good variation since the unripe banana is usually green.

Format your response as:
Component: {component}
Geometry:
1. [variation description]
...
Topology:
1. [variation description]
...
Visual:
1. [variation description]
...

If topology variation is judged infeasible, the prompt instead requests only Geometry and Visual variations.

Prompt D: Scene-Aware Object Editing

Modify the following component of the object: '{component}'.
Make sure to generate a realistic modified version, make sure it looks normal and reasonable like something you would
      usually see in the daily life.
Do not generating variations that are weird or unusual, such as a banana with a hole in it or a banana with a red skin.
Apply this {dimension} variation: {variation_description}.
Keep all other components unchanged.
Generate the image with transparent background.
Make sure the modified object style fits the scene context.

Prompt E: Real-World and Scene Reasonableness Verification

You will see a scene image and a candidate object image. Judge whether the candidate object is reasonable in the real world
      and also reasonable in the given scene. Be strict and practical: reject unusual, implausible, or scene-inconsistent
      objects.

Return format:
RealWorldReasonable: <yes/no>; RealWorldReason: <short reason>
SceneReasonable: <yes/no>; SceneReason: <short reason>
```

Figure F.1: **Object Cousins Prompt Templates.** Prompt templates used in the object cousins generation pipeline include functional decomposition, component-wise variation proposal, scene-aware object editing, and real world/scene reasonableness verification. At runtime, the variables in curly brackets are replaced with dynamic values per object and/or component.

Behind, OnTopOf, Inside]. The selected predicates specify the semantic placement of the object relative to the anchor object. The list of possible predicates for a given task and object is specified by the user and depends on the task. The object is then instantiated based on the predicates. **Note:** multiple predicates can be combined, i.e., an object can be instantiated both **LeftOf** and **InFrontOf** the anchor.

2. **Distractor Objects**: next, a number of distractor objects up to a specified limit can be added to the scene. The distractor objects are selected from the BEHAVIOR [45] dataset of objects and can be filtered based on the following attributes: *mass*, *volume*, *density*, and *object category*. The selected distractor objects are placed in the scene so that they do not collide with existing objects or with each other to maintain physics stability.

F.3. Task Cousins Augmentation

To systematically generate diverse and executable manipulation tasks for reconstructed scene, we employ an automated task proposal pipeline driven by a VLM. This methodology leverages both visual context and structured scene metadata to define realistic tabletop tasks tailored to the specific configuration of each scene.

The pipeline operates through the following steps:

1. **Scene Context Extraction**: For a given scene, the system captures a 2D image for the reconstructed scene in simulation and a list of available interactable objects from the scene's state representation.
2. **Constraint Formulation**: To ensure physical realism and executability, the system incorporates specific robot constraints (e.g., maximum gripper length, single / bimanual arm) and optional object-level constraints (e.g., mandatory inclusion or exclusion of specific items across tasks).
3. **VLM Prompting**: The VLM is provided with the scene image, the filtered object list, the physical constraints, and a predefined set of allowable object predicate states (e.g., OnTop, Inside, Under) in a simulator such as OmniGibson [45] or Isaac Lab [59]. The VLM acts as a robotics expert and proposes a specified number of distinct tasks. Crucially, the VLM is instructed to ensure that each proposed task requires a meaningful state change from the scene's initial configuration.
4. **Configuration Generation**: The VLM outputs structured task definitions, including semantic group mappings and goal conditions formulated as logical predicates. These outputs are parsed and automatically compiled into standardized files that can be utilized directly during data generation as outlined in Appendix H.1.

This automated approach enables the rapid, scalable generation of varied task distributions that are directly compatible with the reconstructed scene by SimFoundry, facilitating extensive data collection and policy evaluation without the bottleneck of manual task engineering.

To ensure reproducibility, we provide the exact prompt template used to query the VLM in Figure F.2. Variables enclosed in curly braces (e.g., `{num_tasks}`) are dynamically populated based on the scene configuration and user constraints at runtime.

F.3.1. Task Cousins Example

First, we record the cluttered scene video and run SimFoundry to generate the reconstructed scene, shown side by side in Fig F.3. We then run the task cousins augmentation pipeline described in Section F.3 to generate 13 proposed tasks in this scene. We then collect 10 demos for each proposed task via human teleoperation and run MimicGen [58] to generate 100 demos for each task. Finally, we use all subsequent demos to finetune $\pi_{0.5}$ and rollout a single multi-task policy in the sim and real setup. The simulation and real-world evaluation results are shown in Table 2. This example demonstrates how SimFoundry's sim-ready scenes could unlock a large variety of tasks and offer a method to scale up multi-task policy training.

VLM Prompt for Task Proposal

```
You are a professional robotic expert in sim-to-real, robot learning, and VLAs.

Look at the provided image of a tabletop scene in a simulation (OmniGibson). Using ONLY the objects listed below, propose
     exactly {num_tasks} distinct tasks that a {robot_type} arm can perform in this scene, ensure that each task goal
     requires a change from the current configuration of the scene, for example, do not propose tasks to put a pear in bowl
     if it is already in the bowl. Each task should have clear goal conditions expressible with object states.

Important:
{robot_constraint}
{object_constraints}

Scene object list (id -> category, name; use these names as group identifiers in predicates):
{object_list}

Output format: for each of the {num_tasks} tasks, output a YAML block that can be merged into a task config. Use this exact
     structure for each task.

For each task provide:
1. task_name: short snake_case name (e.g. stack_cup_on_plate)
2. semantic_group_mapping: map each logical group name you use (e.g. cup, plate) to a list containing exactly one scene
     object "name" from the list above (e.g. cup: [blue_cup_syvtml_19], plate: [teal_plate_qdudop_16]).
3. goal_predicates_all: list of predicates that must ALL be true for success. Each predicate has: state, state_kwargs (null
     if not needed), value (true/false), group, other_group (only for binary states like OnTop, Touching).
4. goal_predicates_any: list of predicates where ANY being true yields success (optional; use null if not needed).

Allowed states (from OmniGibson object_states): {states}

For binary relations use group and other_group. For unary states (Open, ToggledOn, etc.) use group and set other_group to
     null or omit.

Example predicate format:
  goal_predicates_all:
    - state: OnTop
      state_kwargs: null
      value: true
      group: cup
      other_group: plate
  goal_predicates_any: null

Output exactly {num_tasks} tasks. Separate each task with "---" and a task number (e.g. "--- Task 2"). Each task block must
     be valid YAML with keys: task_name, semantic_group_mapping, goal_predicates_all, goal_predicates_any.
```

Figure F.2: **Task Cousins Prompt Template.** The exact prompt template used to query VLM for proposing tabletop manipulation tasks. Bracketed variables are populated dynamically per scene.

(a) Real Scene

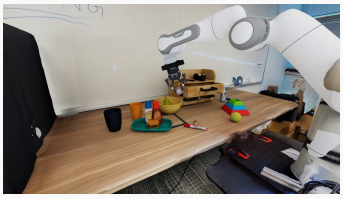
(b) SimFoundry Output

Figure F.3: **Task Cousins Generation Example.** Real and Sim Scene used in task cousins generation example

Task: Place Baseball in Bowl

Instruction: Pick up the baseball, then put it in the bowl.

Goal: Baseball Inside of Bowl



Task: Place Black Eraser on Organizer

Instruction: Pick up the black eraser, then put it on the organizer.

Goal: Black Eraser OnTop of Organizer

Task: Place Burger in Bowl

Instruction: Pick up the burger, then place the burger in the bowl.

Goal: Burger Inside of Bowl

Task: Place Orange Marker on Organizer

Instruction: Pick up the orange marker, then put the orange marker on the organizer.

Goal: Orange Marker OnTop of Organizer

Task: Place Red Bottle in Bowl

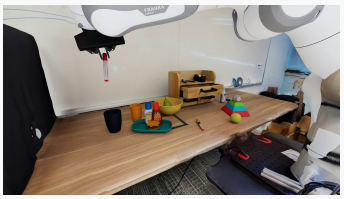
Instruction: Pick up the red bottle, then put the bottle in the bowl.

Goal: Red Bottle Inside of Bowl

Task: Place Red Marker in Cup

Instruction: Pick up the red marker, then put the red marker in the cup.

Goal: Red Marker Inside of Blue Cup



Task: Place Tennis Ball in Bowl

Instruction: Pick up the tennis ball, then put the tennis ball in the bowl.

Goal: Tennis Ball Inside of Bowl

Task: Place Tennis Ball on Organizer

Instruction: Pick up the tennis ball, then put the tennis ball on the organizer.

Goal: Tennis Ball OnTop of Organizer

Task: Place Black Marker on Plate

Instruction: Pick up the black marker, then put the black marker on the plate.

Goal: Black Marker OnTop of Plate

Task: Place Black Eraser in Bowl

Instruction: Pick up the black eraser, then put the black eraser in the bowl.

Goal: Black Eraser OnTop of Bowl

Task: Place Orange Cup in Bowl

Instruction: Pick up the orange cup, then put the orange cup in the bowl.

Goal: Orange Cup OnTop of Bowl

Task: Place Orange Cup on Organizer

Instruction: Pick up the orange cup, then put the orange cup on the organizer.

Goal: Orange Cup OnTop of Organizer

Task: Place Red Marker on Organizer

Instruction: Pick up the red marker, then put the red marker on the organizer.

Goal: Red Marker OnTop of Organizer

G. Detailed Experiment Results

In this section, we present the detailed numbers for experiments in Section 5.

G.1. Detailed Results for Real-to-Sim Policy Evaluation

We present the detailed numbers for policy evaluation in SimFoundry in Table G.1 and the results in PolaRiS [32] in Table G.2. In general, we find that the success rates in SimFoundry align much more closely with those in the real world, with most policies performing poorly in PolaRiS, particularly for tasks with real-world finetuning.

Table G.1: **Real-world vs. simulation success rates (%) for policies evaluated in SimFoundry.** The rightmost columns report per-task Pearson r (higher is better) and MMRV (lower is better) computed between the real-world and simulation success rates across all policies evaluated on that task. Cells marked "–" indicate the policy was not evaluated on that task.

	π_0		$\pi_{0.5}$		GR00T N1.6		GR00T N1.7		DreamZero		Real↔Sim agreement	
Task	Real	Sim	Real	Sim	Real	Sim	Real	Sim	Real	Sim	Pearson r ↑	MMRV ↓
Stack Dishware	100	34	100	64	40	0	–	–	–	–	0.883	0.000
Store Marker	48	4	60	20	32	0	–	–	–	–	0.915	0.000
Throw Away Trash	20	0	48	4	0	0	–	–	–	–	0.910	0.067
Serve Fruits	0	4	72	80	4	20	40	32	8	12	0.960	0.016
Cup in Bowl	88	56	100	92	68	40	92	92	100	92	0.907	0.016
Marker in Cup	40	40	92	88	28	28	88	88	88	80	0.995	0.008
Clear Table	0	12	40	36	0	0	8	28	16	28	0.810	0.016

Table G.2: **Real-world vs. simulation success rates (%) in PolaRiS.**

	π_0		$\pi_{0.5}$		GR00T N1.6		GR00T N1.7		DreamZero		Real↔Sim agreement	
Task	Real	Sim	Real	Sim	Real	Sim	Real	Sim	Real	Sim	Pearson r ↑	MMRV ↓
Stack Dishware	100	0	100	8	40	0	–	–	–	–	0.500	0.200
Store Marker	48	0	60	4	32	0	–	–	–	–	0.822	0.053
Throw Away Trash	20	0	48	0	0	0	–	–	–	–	–	0.253
Serve Fruits	0	4	72	28	4	24	40	4	8	4	0.480	0.288
Cup in Bowl	88	20	100	36	68	76	92	48	100	68	-0.396	0.280
Marker in Cup	40	0	92	4	28	4	88	12	88	4	0.512	0.176
Clear Table	0	0	40	0	0	4	8	0	16	12	-0.037	0.352

G.1.1. Sub-Task Evaluations improve Real-to-Sim Correlations

Sub-task Evaluation Protocol. We introduce a sub-task evaluation protocol that takes advantage of the ability to reset to arbitrary states in simulation. Starting from states where some of the initial sub-tasks are already completed, we evaluate policies on the remaining sub-tasks, allowing a more thorough policy assessment on sub-tasks that occur later in long-horizon tasks. For example, for **Store Marker**, we start with the cabinet drawer already open and evaluate whether the policy can complete the remaining sub-tasks. The policy success rate with initial subtasks completed in sim is compared with the full end-to-end task success rate in the real world which we found improved evaluation correlations between sim and real.

Sub-task evals improve correlations and provide insights into failure modes. Evaluating from states with completed sub-tasks can improve correlations for long-horizon fine-tuned tasks and expose failure modes, providing actionable insights for policy improvement. As seen in Table G.3, the mean pearson correlation improves from 0.902 to 0.951 with sub-task evaluations on the fine-tuned tasks. For example, in the **Store Marker** task, once the drawer is opened, $\pi_{0.5}$ can almost always complete the rest of the task, in both sim and real.

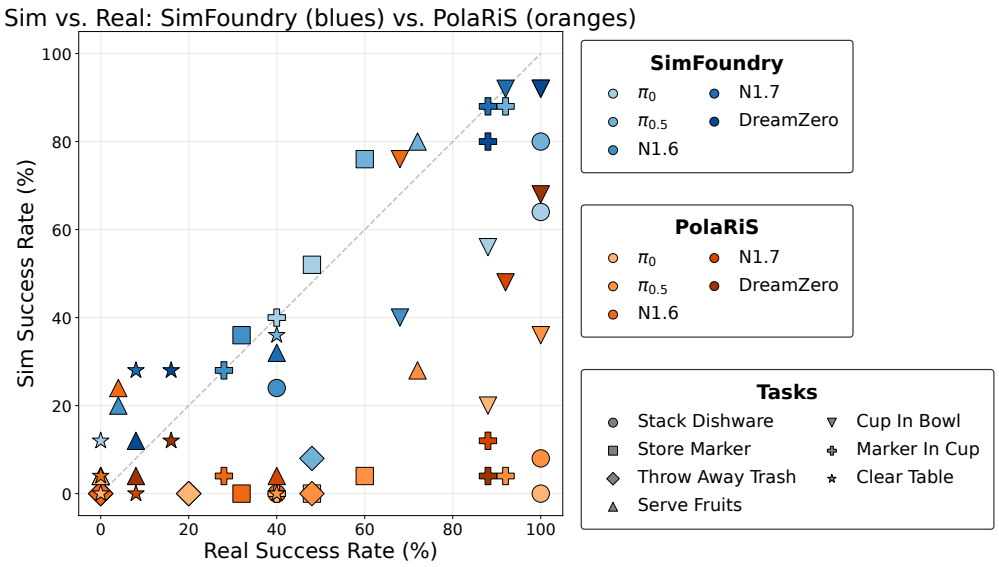


Figure G.1: **Enlarged Real-to-Sim policy evaluation correlations with task labels.** This figure expands the right panel of Figure 4. Each point compares real-world and simulated success rates for a policy-task pair. The dashed diagonal indicates perfect agreement between simulated and real-world success rates. Blue points correspond to SimFoundry evaluations and orange points to PolaRiS evaluations. SimFoundry points lie closer to the diagonal across tasks, matching the higher Pearson correlations and lower MMRV values reported in Table G.1 and Table G.2.

Table G.3: **Real-world vs. simulation success rates (%) correlations improve when evaluating on sub-tasks.**

	π_0		$\pi_{0.5}$		GR00T N1.6		Real↔Sim agreement	
Task	Real	Sim	Real	Sim	Real	Sim	Pearson $r \uparrow$	MMRV $\downarrow$
Stack Dishware	100	64	100	80	40	24	0.961	0.000
Store Marker	48	52	60	76	32	36	0.981	0.000
Throw Away Trash	20	0	48	8	0	0	0.910	0.067

G.2. Detailed Results for Sim-to-Real Experiments

The per-task success rates for the object cousin experiments are presented in Table G.4. The models are trained with either - **Twin**, i.e., only on the digital twin object, or **+ 9 cousins**, i.e., data generated with the twin plus 9 cousin objects. The different eval settings are:

1. Sim Twin: evaluation on the reconstructed twin object(s).
2. Sim Cousins: evaluation on a held-out digital cousin(s) of the reconstructed twin. This is an object for which data is not collected in either sim or the real world.
3. Real Twin: evaluation on the real-world object(s) that were reconstructed in simulation.
4. Real Cousin: evaluation on held-out real-world object(s).

Ablation of Object, Scene and Task Cousins. Figure G.2 summarizes how different forms of SimFoundry-generated data diversity affect policy performance. Across object, scene, and task cousins, we find that structured variation consistently improves policy robustness beyond training on the reconstructed twin alone. Object cousins improve instance-level generalization by exposing policies to affordance-preserving changes in object geometry, appearance, and topology, yielding an average task success improvement of 17% during zero-shot sim-to-real transfer. Scene cousins target a complementary form of generalization by varying semantic object relations and introducing layout diversity; these improve task success by an average of $\sim 13\%$ on the

twin scene and $\sim 29\%$ on cousin scenes. Task cousins provide behavioral and action diversity by adding related demonstrations that share objects, predicates, or intermediate behaviors with the target task; this yields the largest average improvement of 40%.

In addition to these cousin-based augmentations, Figure G.2 also shows that SimFoundry data can effectively complement limited real-world demonstrations through co-training. While zero-shot policies trained only on SimFoundry data already transfer to the real world, adding real data further improves performance in most DROID settings.

YAM

		Twin	+ 9 Cousins
Stack Dishware	Sim Twin	83	92
	Sim Cousins	43	66
	Real Twin	39	43
	Real Cousins	21	42
Pot On Stove	Sim Twin	85	100
	Sim Cousins	17	93
	Real Twin	91	99
	Real Cousins	14	64
Throw Away Trash	Sim Twin	97	97
	Sim Cousins	97	94
	Real Twin	0	28
	Real Cousins	2	8

DROID

		Twin	+ 9 Cousins
Stack Dishware	Sim Twin	80	88
	Sim Cousins	64	92
	Real Twin	88	96
	Real Cousins	88	100
Store Marker	Sim Twin	20	60
	Sim Cousins	8	28
	Real Twin	4	20
	Real Cousins	4	4
Throw Away Trash	Sim Twin	8	48
	Sim Cousins	4	48
	Real Twin	0	20
	Real Cousins	0	8

Table G.4: **Policy Robustness Using Object Cousins.** Across multiple robot embodiments and multiple tasks, leveraging additional object cousins [17] improves direct sim2real policy transfer on the original target scene objects and additional held-out unseen objects.

G.3. Detailed Object Cousin Ablation

To better understand the importance of object cousins, we run an additional ablation on our set of bimanual tasks, testing sim2real zero-shot performance over training datasets that include only the reconstructed digital twin scene objects, or additionally including 1, 3, or 9 object cousins as part of the training mix. All runs used a fixed 1000 demonstration bandwidth and were split evenly among each scene objects' instance. Results are aggregated over 25 eval trials. To further guarantee reproducibility, we randomly sample each pose initialization and deterministically align them in both sim and real, as done in our real-to-sim policy evaluation setup (see Appendix J for more details).

Our results are shown in Table G.8. We find in general that increasing the number of object cousins tends to improve zero-shot sim2real policy transfer, both on the original twin scene objects and held-out unseen scene objects, highlighting the potential for increasing object cousins to reliably improve policy robustness.

Table G.8: **Sim-to-real policy training results across object cousins.**

YAM

		Twin	+ 1 Cousin	+ 3 Cousins	+ 9 Cousins
Stack Dishware	Sim Twin	83	89	100	92
	Sim Cousins	43	44	65	66
	Real Twin	39	41	37	43
	Real Cousins	21	32	27	42
Pot On Stove	Sim Twin	85	100	93	100
	Sim Cousins	17	27	35	93
	Real Twin	91	100	94	99
	Real Cousins	14	38	16	64
Throw Away Trash	Sim Twin	97	98	98	97
	Sim Cousins	97	100	97	94
	Real Twin	0	9	45	28
	Real Cousins	2	17	14	8

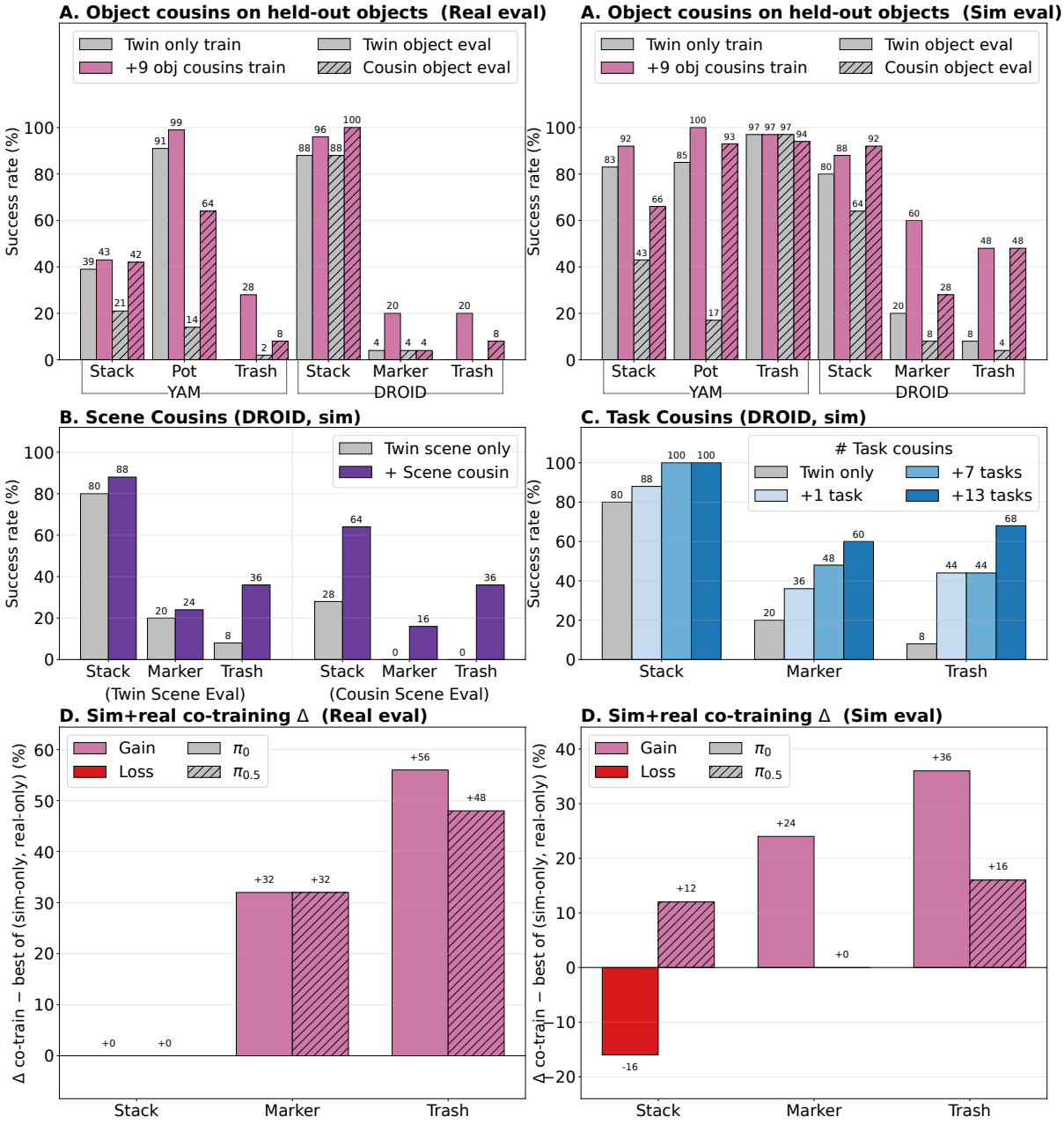


Figure G.2: **SimFoundry data diversity along different axes scales data generation and policy performance.** We ablate how different sources of SimFoundry-generated data improve policy learning and generalization. **(A)** Object cousins improve robustness across DROID and YAM by training policies on affordance-preserving object variants, yielding an average zero-shot sim-to-real success improvement of 17% and up to a 50% real-world gain on held-out **Pot on Stove** objects. **(B)** Scene cousins improve layout generalization by training on semantically modified object arrangements, producing an average improvement of 21%, including a 28% gain on **Throw Away Trash** in the twin scene and 16% success on held-out **Store Marker** cousin layouts where the twin-only policy achieves 0%. **(C)** Task cousins improve downstream task learning by adding demonstrations from related tasks while keeping the total number of demonstrations fixed; 13 task cousins improve **Throw Away Trash** by 60% and **Store Marker** by 40% in simulation. **(D)** Sim-and-real co-training further improves performance by combining scalable SimFoundry demonstrations with limited real data, increasing $\pi_{0.5}$ real-world **Store Marker** success from 60% to 92% and improving π_0 simulated **Throw Away Trash** success by 36%. Together, these results show that SimFoundry provides complementary forms of structured data diversity across objects, layouts, tasks, and real/sim data mixtures.

Table G.5: **Boosting Performance with Scene Cousins (DROID, simulation)** Success rates shown below.

	twin only	+ scene cousin
Stack Dishware	80	**88**
Stack Dishware- cousin	28	**64**
Store Marker	20	**24**
Store Marker- cousin	0	**16**
Throw Away Trash	8	**36**
Throw Away Trash- cousin	0	**36**

Table G.6: **Boosting Performance with Task Cousins.** Adding additional tasks and cousins with the same or similar objects increases performance for the downstream task.

	twin only	+1 task	+7 tasks	+13 tasks
Stack Dishware	80	88	**100**	**100**
Store Marker	20	36	48	**60**
Throw Away Trash	8	44	44	**68**

Table G.7: **Co-training with SimFoundry generated data.** We compare success rates of models trained with simulation-only data (-**S**) to those trained with real-world demos (-**R**) as well as combinations of both simulation and real data (-**co-train**). Each model type is evaluated in both the real-world scene (-Real) and the SimFoundry reconstruction (-Sim).

	π_0-**S**	π_0-**R**	π_0-**co-train**	$\pi_{0.5}$-**S**	$\pi_{0.5}$-**R**	$\pi_{0.5}$-**co-train**
Stack Dishware- Sim	92	34	76	88	64	100
Stack Dishware- Real	96	100	100	96	100	100
Store Marker- Sim	16	4	40	60	20	60
Store Marker- Real	4	48	80	20	60	92
Throw Away Trash- Sim	0	0	36	48	4	60
Throw Away Trash- Real	0	20	76	20	48	96

H. Policy Details

H.1. Data Generation Details

SimFoundry leverages a combination of human-collected demonstrations and automated data augmentation to generate synthetic datasets useful for training robot learning policies.

For a given task, we first collect a small number ($\sim 10 - 15$) of demonstrations via human operator-controlled JoyLo [35] systems. Then, we augment those demonstrations using MimicGen [58], both increasing the trajectory diversity (via demonstration count) as well as visual diversity by applying domain randomization: material randomization, camera pose randomization, and (specifically in the DROID setup) table height randomization. The resulting datasets are used to train robot learning policies that can be deployed zero-shot in the real world.

H.2. Policy Training Details

For the DROID sim-to-real experiments, we finetune the DROID-pretrained joint-position versions of π_0 and $\pi_{0.5}$. Each policy is trained with a batch size of 256, a learning rate of $1e-5$, and for 10k gradient steps. In simulation, the policies are evaluated every 1k steps, and the best-performing checkpoint is evaluated in the real world. We then report the best-performing checkpoint for each model.

For the YAM-based tasks, we trained flow-matching policies. Observations included joint state proprioception, top-down fixed camera RGB images, and per-wrist camera RGB images. The policy action space consisted of N-DOF joint position commands and a normalized 1-DOF continuous gripper open / close command. We train for 40k steps and similar to DROID, run sim evals periodically, selecting the best-performing checkpoints to evaluate zero-shot in the real-world.

Real-to-Sim Policy Details and Selection. For the real-to-sim experiments, we once again finetune the DROID-pretrained joint-position checkpoints of π_0, $\pi_{0.5}$ and GR00T N1.6 [61] for the following tasks - **Stack Dishware**, **Store Marker**, **Throw Away Trash**. As previously, policies are evaluated every 1k steps and the best-performing checkpoint is evaluated in the real world. For the simpler tasks - **Cup in Bowl**, **Marker in Cup**, **Serve Fruits**, and **Clear Table**, the following additional policies are also deployed zero-shot without any finetuning: GR00T N1.7and DreamZero [100] in addition to the pretrained checkpoints of the first three. All models output joint positions and gripper positions as actions.

I. Robot Platform and Task Details

I.1. Robot Embodiments.

We focus on two robot embodiments – the DROID [39] platform, and a YAM workcell [69]. The DROID platform consists of a single Franka Panda robot arm, left and right external ZED-2 cameras, and a wrist-mounted ZED-Mini camera, with the robot and external cameras mounted to a portable standing desk. For data collection, we use an Oculus VR headset to teleoperate the robot. In both simulation and the real world, a joint-position controller is used during policy rollout, with the gains tuned higher in simulation to minimize the tracking error.

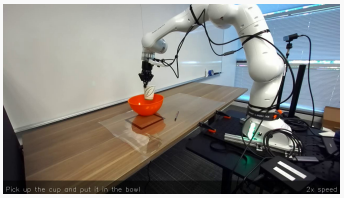
The YAM workcell consists of a bimanual manipulator, a cage, a wrist-mounted RealSense D405 camera per arm, and an external top-down view camera. We use JoyLo [35] to teleoperate the YAM arms, and a joint-position controller is used during both data collection and policy evaluation.

I.2. Task Rubric

In this sub-section, we provide the scoring rubric for each task, along with the language instruction provided to the VLAs. All sub-tasks need to be completed for a task to be successful. For our experiments, we mainly use binary success where the whole task is completed successfully, except for the sub-task evaluation protocol described in Appendix G.1.1.

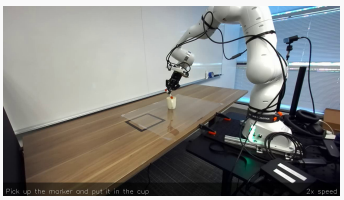
Task: Cup in Bowl

Instruction: Pick up the cup, and put the cup inside the bowl.
Rubric: 1. Touch cup 2. Pick cup 3. Place in bowl



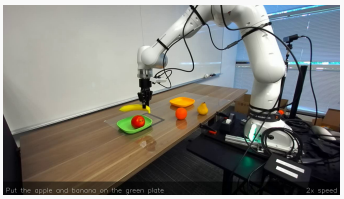
Task: Marker in Cup

Instruction: Pick up the marker, and put the marker in the cup.
Rubric: 1. Touch marker 2. Pick up marker 3. Place marker in cup



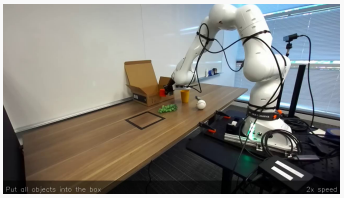
Task: Serve Fruits

Instruction: Put both the banana and the apple on the green plate.
Rubric: 1. Pick up banana 2. Pick up apple 3. Place banana on green plate 4. Place apple on green plate



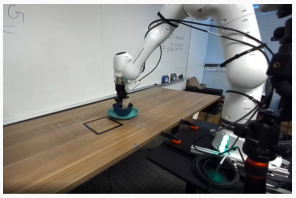
Task: Clear Table

Instruction: Put all objects in the brown box
Rubric: 1. Place cube in box 2. Place grapes in box 3. Place cup in box 4. Place marker in box 5. Place baseball in box



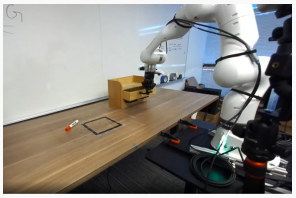
Task: Stack Dishware

Instruction: Put the bowl on the plate, then put the cup in the bowl.
Rubric: 1. Pick up bowl 2. Place bowl on plate 3. Pick up cup 4. Place cup in bowl



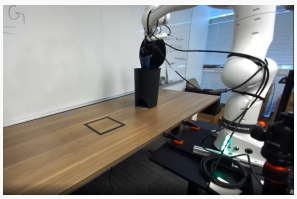
Task: Store Marker

Instruction: Open drawer, pick up marker, place marker in drawer, then close drawer.
Rubric: 1. Open cabinet drawer 2. Pick up marker 3. Place marker in drawer 4. Close drawer



Task: Throw Away Trash

Instruction: Open the trash can, pick up the cup, put the cup into the trash can, and then close the trash can.
Rubric: 1. Open trash can 2. Pick up cup 3. Place cup inside trash can 4. Close trash can



Task: Pot on Stove

Instruction: Put the pot on the stove
Rubric: 1. Left arm grasps pot handle 2. Right arm grasps pot handle 3. Pot is lifted 4. Pot is placed on Stove

J. Real-to-Sim Policy Evaluations

J.1. Evaluation Protocol

For real-to-sim evaluations, we use a standardized protocol to maintain fairness and minimize variance between runs. For each task, we run 25 rollouts and each of the objects has a defined spatial reset distribution. The spatial distribution for each object is uniformly divided into a 5-by-5 grid, yielding 25 positions per object. For each rollout, one of the 25 positions is sampled independently per object, without replacement, and the center of the object is placed at this position. We also sample a rotation for each object per position, and these positions are held fixed across all checkpoints for a specific task. Figure J.1 is an example of a task grid, detailing 25 possible starting positions for each object, each one represented with a dot.

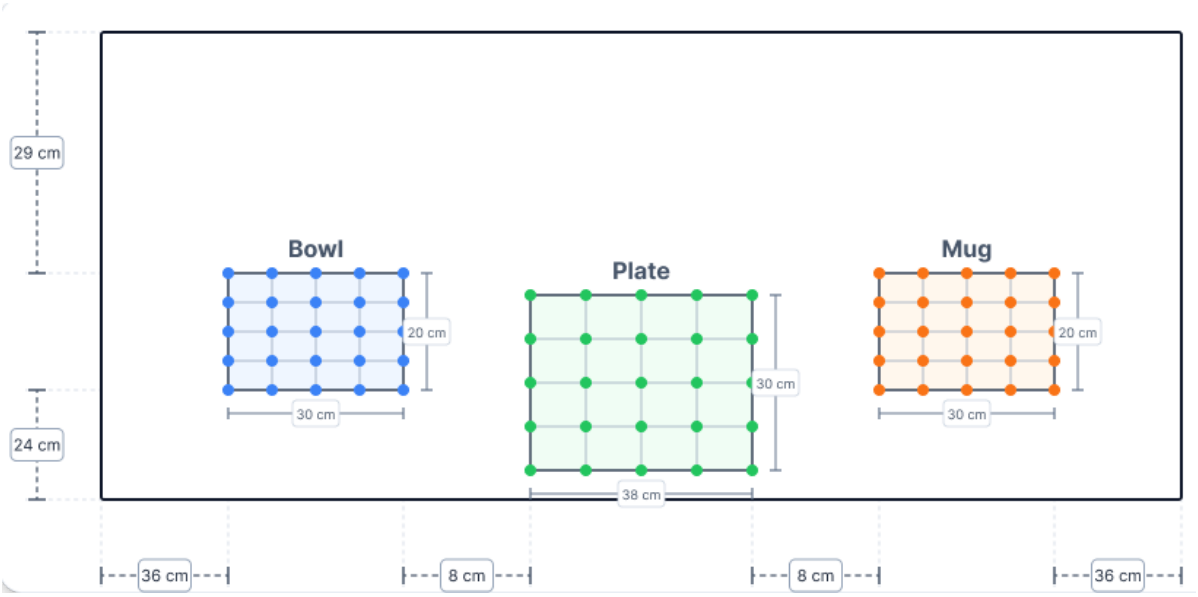


Figure J.1: **Stack Dishware Evaluation Grid.** Task grid diagram detailing 25 initial starting positions of each object in **Stack Dishware**. The outer box represents the tabletop itself. Not to scale.

The ranges of the positions are matched between the SimFoundry and real-world scenes but the exact positions may not always correspond. This was done intentionally to get a more distributional correspondence between simulation and the real world and to prevent overfitting takeaways and correlations to proprioceptive robot states.

J.2. Metrics

Task success is our main metric, and for each evaluation a policy earns either a 0 or a 1 depending on if it fully accomplished the task it is being evaluated on. From task success we calculate the following metrics.

Real-to-Sim evaluation metrics. To quantify how well simulation-based evaluations predict real-world policy performance, we follow prior work [32, 47] and report two complementary metrics: Pearson correlation and Mean Maximum Rank Violation (MMRV). Let $\Pi = \pi_1, \ldots, \pi_N$ denote the set of evaluated policies. For each policy π_i, let $x_i \in [0, 1]$ denote its real-world score and $y_i \in [0, 1]$ denote its corresponding simulation score, computed using either task success rate or normalized reward. We collect these scores into vectors $\mathbf{x} = (x_1, \ldots, x_N)$ and $\mathbf{y} = (y_1, \ldots, y_N)$.

The Pearson correlation coefficient measures whether simulation preserves linear trends in real-world perfor-

mance:

$$\rho(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2}}, \tag{1}$$

where $\bar{x} = \frac{1}{N}\sum_i x_i$ and $\bar{y} = \frac{1}{N}\sum_i y_i$. Larger values indicate stronger agreement between simulated and real-world performance, with $\rho = 1$ corresponding to perfect positive linear correlation.

Pearson correlation captures score-level agreement, but it does not directly measure whether simulation preserves policy rankings. We therefore also compute Mean Maximum Rank Violation (MMRV), which measures the average magnitude of the largest real-world performance gap involved in a simulation-induced ranking error. For each policy π_i, we identify policies π_j whose ordering relative to π_i differs between simulation and the real world, and take the largest real-world score difference among such inversions:

$$\mathrm{MMRV}(\mathbf{x}, \mathbf{y}) = \frac{1}{N}\sum_{i=1}^{N} \max_{j \in \{1,\ldots,N\}} \left[|x_i - x_j| \cdot \mathbb{1}\left(\mathbb{1}[y_i < y_j] \neq \mathbb{1}[x_i < x_j]\right) \right]. \tag{2}$$

Here, $\mathbb{1}(\cdot)$ is the indicator function. Intuitively, MMRV penalizes cases where simulation ranks two policies differently from the real world, with larger penalties assigned when the mis-ranked policies differ substantially in real-world performance.

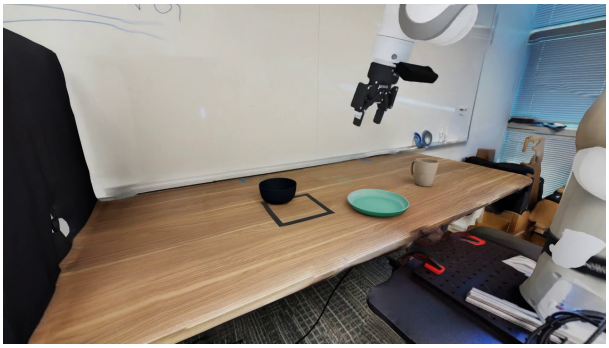
J.3. PolaRiS Real-to-Sim Experiment Details

PolaRiS [32] acts as a state-of-the-art baseline for evaluating real-world policies in simulation by providing a browser-based environment composer for reconstructing digital scenes that can then be brought up in Isaac Sim and evaluated using generalist policies such as $\pi_{0.5}$. These scenes are constructed around the DROID setup which acts as an anchor for the user to set asset initial positions and variations to create PolaRiS-ready environments for export. We reconstructed our experiment scenes using PolaRiS's custom environment creation pipeline and tested to see how the simulated evaluation success rates correlated to real-world success rates, showing SimFoundry maintains significantly higher correlation as seen in Figure 4.

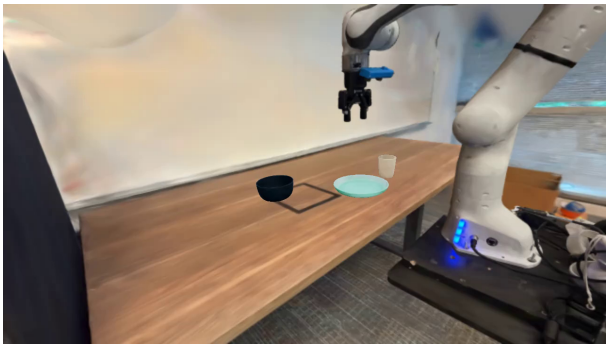
J.3.1. PolaRiS Custom Environment Creation

It is important to note that PolaRiS only provides the environment composer for scene reconstruction, meaning users must utilize recommended external software to obtain background and object digital reconstructions. For our PolaRiS experiments, we reconstructed our DROID environment by first obtaining a video scan of the background and running it through COLMAP to obtain a sparse-reconstruction dataset of the scene. 2DGS was then used to obtain a corresponding splat and mesh of the environment, which were imported into the PolaRiS environment composer. SimFoundry object assets were used in conjunction with the 2DGS output to recreate full task environments for some of our real-to-sim DROID tasks.

To recreate our 25 unique asset starting positions, manual human effort in the environment composer is required to set asset initial conditions relative to the DROID setup. This includes scaling, transforming, and rotating objects until they match the initial conditions used for real-world evaluations. This process took about 15 minutes for each scene (assuming background and object assets are already supplied). The resulting reconstruction is visible in Figure J.2.



(a) SimFoundry scene



(b) PolaRiS scene

Figure J.2: **SimFoundry and PoLaRiS Scene Comparison.** SimFoundry scene compared with a PolaRiS scene for the task **Stack Dishware**

J.3.2. PolaRiS Modification

Further modifications were required in order for our custom PolaRiS environments to be properly loaded into Isaac Sim and evaluated using the PolaRiS evaluation pipeline. The PolaRiS evaluation code was not immediately compatible with the exported custom environments, requiring code changes to ensure that every asset gets loaded into the simulator with its textures properly and in the defined initial positions. The collision physics also were not being properly applied to our scene, creating an issue where objects would immediately fall through the table. An invisible collider plane was added inline with our tabletop mesh to act as a flat surface that objects can rest on. This was especially necessary for tasks with objects that can roll away, such as **Marker in Cup** where the marker asset was subject to the slightly uneven mesh obtained during environment digital reconstruction. We also needed to manually tune the marker asset mass so that its physics dynamics more closely aligned with the real-world. Articulated assets are not supported by PolaRiS, but we were able to modify existing code so that they were loaded into the sim with the required physics allowing for evaluation of articulated tasks such as **Store Marker**. To obtain the same external viewpoint for policy observations in simulation as seen in Figure J.2, we needed to patch exported custom scenes manually in order to set the camera at the orientation matching real-world and SimFoundry evaluations.

J.3.3. Real-to-Sim Policy Evaluations in PolaRiS

Generalist policy evaluations were conducted in PolaRiS in exactly the same manner as our real-world and SimFoundry evaluations. We evaluated π_0, $\pi_{0.5}$, GR00T N1.6, GR00T N1.7, and DreamZero in PolaRiS. π_0-Finetune, $\pi_{0.5}$-Finetune, and GR00T N1.6-Finetune checkpoints were also evaluated on their respective tasks. Tasks evaluated in PolaRiS are **Cup in Bowl**, **Marker in Cup**, **Serve Fruits**, **Stack Dishware**, **Store Marker**, and **Throw Away Trash**; the details of which can be found in Appendix I.2.

J.3.4. PolaRiS Results Analysis

Overall, PolaRiS yielded a low correlation to real-world policy evaluation across all tasks evaluated in our custom scene. As seen in Figure 4 policies evaluated in PolaRiS consistently underperformed compared to their real-world success rates. PolaRiS provides an $\pi_{0.5}$ policy cotrained on 10% PolaRiS simulation data and 90% DROID data at 1000 steps. Evaluating this policy in our custom scenes yielded higher success rates than $\pi_{0.5}$, suggesting that simulation data co-training is essential for high correlation in PolaRiS. We then attempted to evaluate the $\pi_{0.5}$-PolaRiS cotrained policy in the real-world, but its motions were too exaggerated, causing the DROID arm to make unsafe motions that could have potentially harmed itself or the end effector, so we did not continue.

K. Human Interaction

K.1. Human Intervention Details

In addition to being fully automated, we provide a unified GUI with accessible touchpoints that allow human operators to easily tune our pipeline's intermediate outputs. For example, during the scene decomposition process, a human operator can intervene and enforce specific constraints on individual objects being extracted, and can quickly tweak the generated pose and scale of meshes generated during the generation process.

K.2. Interactive Pose Refinement

The Extraction stage estimates a per-object similarity transform—3-DoF translation, 3-DoF rotation, and an isotropic scale that registers each generated canonical mesh to the metric scene reconstructed pointclouds.

Initialization from the automatic estimate. The interactive tool starts with the transform emitted by the automatic Pose Matching stage, so the user always begins from the best machine estimate rather than from scratch; in the common case where the automatic pose is already correct, no edits are needed. Objects are processed one at a time. For each object we load its canonical mesh and reconstruct the scene point cloud from the stage's estimated depth and camera intrinsics K (the same per-object depth and inpainted-RGB frames used by Pose Matching), so that the editing context is identical to the geometry the automatic estimate was fit against.

Overlay visualization. We overlay the scene pointclouds with target object mesh, so that the user can inspect the overlay visually and use the pointcloud as reference. Because a dense point cloud can occlude the mesh and obscure these cues, the tool provides two viewing aids: the ability to toggle the point cloud visibility, and the ability to dynamically modify its density, enabling the user trade-off scene context with an unobstructed view of the mesh without altering the underlying estimate.

Manual Adjustment The user adjusts objects' 6D poses and scales through keyboard commands. The editor also enables adjustable translation and rotation step sizes so the user can dynamically transition from coarse to fine-grained alignment within a single session. Once satisfied, the user saves the pose, after which the tool serializes the final transform consumed by the downstream pipeline so that the manually-refined pose is loaded in place of the automatic estimate on all subsequent launches with no further intervention.

Iterative refinement across sessions. Saved poses are written to a fresh output so that no edit overwrites the automatic estimate or a previous manual pass. The tool can therefore be re-entered to resume from either the automatic output or any earlier interactive pass, refining the same scene over multiple sittings while retaining every intermediate version for comparison or rollback. In practice this makes the tool optional: the automatic pipeline runs unattended and suffices for the bulk of objects, while the small number of poses that matter most for the scene or a quantitative evaluation can be improved with a few minutes of guided manipulation per object. See Figure K.1 for an actual example of running interactive pose refinement.

Use Cases. We primarily use iterative pose refinement for more accurately localizing and aligning the poses of the objects for the Real-to-Sim experiments (Section 5.1). For Sim-to-Real data generation, iterative refinement has less utility, as long as the ranges of spatial initializations for the objects in simulation are large enough to cover the real-world test scenarios.

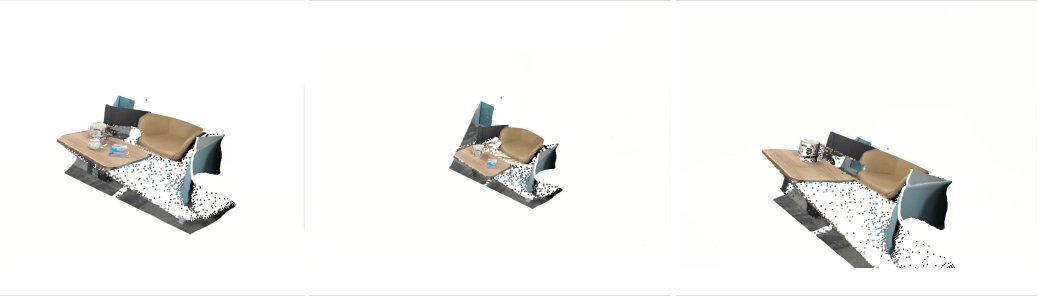
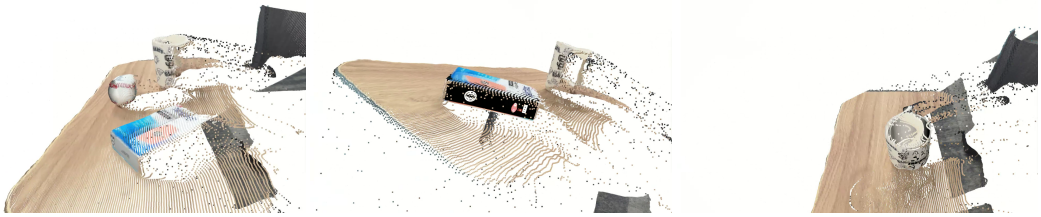
Step	1	2	3
Object Adjust	Baseball	Box	Mug
Full Scene Point-cloud			
Object Mesh & PC Overlay			

Figure K.1: **Interactive Scene Editor Procedure.** The interactive scene editor launches the adjusted object mesh and the (inpainted) dense pointcloud for the scene, allowing the user to adjust object poses to align to the scene pointcloud. This process continues until all object poses have been adjusted. Note that in every step the GUI loads an inpainted pointcloud that erases previous objects to support occluded object pose tuning.

L. System Analysis

L.1. 3D Reconstruction Evaluation Details

We describe (a) how quasi-ground truth scene reconstructions are obtained, (b) how SAM3D [74] outputs are placed in a common frame so that they can be compared with SimFoundry outputs, and (c) quantitative and qualitative reconstruction results across the full set of 12 scenes.

We categorize 12 table-top scenes from difficulty low to high based on the object occlusion level using objects from the YCB dataset [9], as shown in Table L.1.

L.1.1. Quasi-Ground-Truth Scene Reconstruction

Evaluating reconstruction fidelity in cluttered scenes requires per-object ground truth poses, which are difficult to recover directly under heavy occlusion. We therefore stage each benchmark scene incrementally and record a quasi-ground truth pose for each object while it is still fully visible. Concretely, we place the object furthest in the background first and infer its 6-DoF pose with FoundationPose [89] and the known ground truth CAD mesh of the object from an unoccluded view, then repeat the procedure for each subsequent object until the full scene is staged. The recorded poses serve as the ground truth for all subsequent metrics. At evaluation time, only the final fully-staged (and therefore occluded) scene image is provided to either reconstruction method.

L.1.2. SAM3D Reconstruction Pipeline

SAM3D produces per-object meshes and relative object poses in its own coordinate convention and with an arbitrary global scale, so a direct comparison with SimFoundry requires placing both outputs in a shared metric world frame. We use SAM3D solely for object geometry and object relative poses, and resolve the remaining frame and scale ambiguities through three steps:

1. **Convert coordinate conventions**: We map SAM3D mesh coordinates through the required axis-system changes (GLB convention $\rightarrow$ SAM3D internal $\rightarrow$ OpenCV camera convention) so the object points are expressed in the same camera convention used by our pipeline.
2. **Place objects in a common world frame**: We apply the same world transform used by SimFoundry so SAM3D objects are moved into the identical metric world frame (same origin/z-axis convention).
3. **Resolve SAM3D global scale ambiguity**: Because SAM3D reconstruction scale is arbitrary, we estimate one scene-level scale by matching SAM3D's merged scene point cloud with ground-truth point cloud by calculating the smallest chamfer distance. Specifically, SAM3D's output point clouds are scaled around the camera origin, to stay physically consistent with camera-frame scaling.

After these steps SAM3D outputs live in the same metric world as SimFoundry outputs, so any residual differences in the reported metrics are attributable to reconstruction quality rather than to frame or scale mismatch.

L.1.3. Quantitative Reconstruction Results

With output from SimFoundry and SAM3D, we measure reconstruction fidelity via three 3D geometric metrics: Chamfer Distance, F1-Score (with threshold 0.01 meters) and Object Bounding Box Position Error against quasi-ground-truth, the quantitative result is shown in Table L.2. We find that SimFoundry outperforms SAM3D in zero-shot, and additionally further improves its geometric reconstruction fidelity with a few minutes of interactive human iteration.

L.1.4. Qualitative Reconstruction Results

We provide the corresponding visual reconstructions results for all 12 benchmark scenes in Table L.3 below, covering the full range of clutter and occlusion levels used in the evaluation.

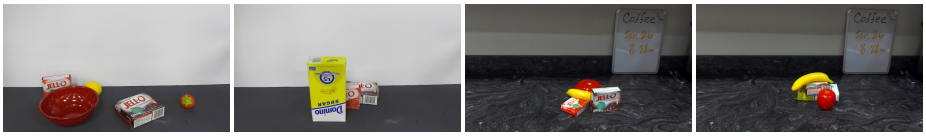
Difficulty	Input Scenes
Easy	
Med	
Hard	

Table L.1: **Reconstructed Scenes.** Categorized by Easy (No Occlusion), Mid (Slight Occlusion), and Hard (Strong Occlusion).

Table L.2: **Scene Reconstruction Quantitative Results.** Entries show Average ± Standard Deviation.

Difficulty	Metric	SAM3D Zero Shot	SimFoundry Zero Shot	SimFoundry Tuned (3min/Obj)
Easy	Chamfer Dist (m) ↓	0.0081 ± 0.0024	0.0042 ± 0.0013	$\mathbf{0.0026 \pm 0.00026}$
	F1 Score ↑	0.71 ± 0.15	0.92 ± 0.071	$\mathbf{0.99 \pm 0.0069}$
	Pos Error (m) ↓	0.016 ± 0.0058	0.0060 ± 0.0019	$\mathbf{0.0041 \pm 0.00037}$
Medium	Chamfer Dist (m) ↓	0.0087 ± 0.0028	0.0047 ± 0.0012	$\mathbf{0.0033 \pm 0.00068}$
	F1 Score ↑	0.66 ± 0.18	0.87 ± 0.089	$\mathbf{0.97 \pm 0.026}$
	Pos Error (m) ↓	0.018 ± 0.0067	0.0076 ± 0.0038	$\mathbf{0.0057 \pm 0.0030}$
Hard	Chamfer Dist (m) ↓	0.0088 ± 0.0022	0.0091 ± 0.0076	$\mathbf{0.0039 \pm 0.0013}$
	F1 Score ↑	0.68 ± 0.14	0.81 ± 0.071	$\mathbf{0.93 \pm 0.049}$
	Pos Error (m) ↓	0.022 ± 0.010	0.018 ± 0.018	$\mathbf{0.0073 \pm 0.0022}$

L.2. Comparison between Manual and Automatic Background Pipeline

In this section, we compare the quantitative and qualitative reconstruction results between the manual background pipeline and automatic background pipeline mentioned in Appendix E.5.

L.2.1. Qualitative Reconstruction Results

We record five in-the-wild scenes and run both background reconstruction pipelines. The side-by-side visual results are shown in Table L.5. The automatic background pipeline can produce floating artifacts around the support surface that partially occlude foreground objects, which may occur when the object-removal model hallucinates content during image inpainting.

L.2.2. Quantitative Reconstruction Results

We quantify the agreement between each rendered scene shown in Table L.5 and the corresponding real video frame using 7 complementary metrics as follows:

- **PSNR** (Peak Signal-to-Noise Ratio, dB, ↑) is defined as $10 \log_{10}(1/\text{MSE})$ on intensities normalized to $[0, 1]$ and measures global pixel-wise fidelity; because it derives from the mean squared error, it is dominated by a small number of large deviations and is sensitive to global misalignment.
- **SSIM** (Structural Similarity Index, $[-1, 1]$, ↑) compares local luminance, contrast, and structure over sliding windows, capturing perceived structural agreement while remaining comparatively tolerant of

uniform intensity shifts.

- **MAE** (Mean Absolute Error, ↓) is the mean of $|I_{\text{render}} - I_{\text{GT}}|$ over all pixels and channels, reporting the average per-pixel intensity deviation.
- **RMSE** (Root Mean Squared Error, ↓) is the square root of the mean squared error and penalizes large per-pixel errors more heavily than MAE.
- **NCC** (Normalized Cross-Correlation, $[-1, 1]$, ↑) computes the correlation between the zero-mean, unit-variance render and ground-truth images; by factoring out global brightness and contrast it isolates spatial/structural alignment, making it the most direct indicator of whether the background geometry is registered to the real footage (values near zero indicate near-random alignment).
- ΔE (CIE76 color difference, ΔE^*_{ab}, ↓) is the mean Euclidean distance between the images in the perceptually-uniform CIELAB color space, quantifying color/appearance error in units calibrated to human color perception.
- **EdgeMAE** (Sobel edge MAE, ↓) is the mean absolute difference between the Sobel gradient-magnitude maps of the two images, measuring how well structural edges and contours coincide independently of absolute color.

Together these metrics span pixel fidelity (PSNR, MAE, RMSE), structural and perceptual similarity (SSIM, EdgeMAE), color accuracy (ΔE), and geometric alignment (NCC), providing a comprehensive comparison. We evaluate each metric on 50 uniformly sampled frames per scene across five scenes, and report the per-scene means together with their cross-scene average in Table L.6.

The automated pipeline outperforms the manual, hand-curated background reconstruction across all reported metrics. One potential interpretation of this result is that the automatic pipeline does not estimate the background alignment but derives it: the background-to-world transform is composed analytically from the same camera poses that define the evaluation viewpoints, so the reconstruction is registered to the ground-truth frame by construction and stays consistent across views. Manual alignment instead approximates this six-degree-of-freedom transform by human eye, and since reprojection error scales with rotation error and scene depth, small imperceptible offsets compound into large pixel misalignment. This is clearest in NCC, which isolates spatial alignment; the PSNR, RMSE, and ΔE gains follow from the same tighter registration.

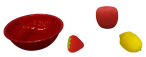
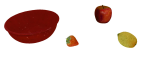
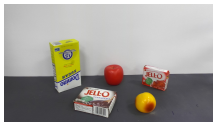
Scene Name	Difficulty	Real Scene	SAM3D Zero Shot	SimFoundry Zero Shot	SimFoundry Tuned (3min/obj)
Desk 1	Easy				
Desk 2	Easy				
Desk 3	Easy				
Kitchen 1	Easy				
Desk 4	Medium				
Desk 6	Medium				
Desk 8	Medium				
Desk 9	Medium				
Desk 5	Hard				
Desk 7	Hard				
Kitchen 2	Hard				
Kitchen 3	Hard				

Table L.3: **Scene Reconstruction Qualitative Results.** We compare SimFoundry against SAM3D across Easy, Medium, and Hard scenes.

N Objects	Time (min)	Original	Recon. Twin	Cousins Image
2	19.98			
4	34.07			
9	41.36			
10	42.76			
10	49.02			
10	51.01			
10	51.39			
10	58.71			
11	67.13			

Table L.4: **Additional real-to-sim reconstruction results.** We show additional real-world input images, the corresponding reconstructed digital twins generated by SimFoundry, and sampled digital cousin scene variations. The first two columns report the number of reconstructed objects and the total wallclock time to reconstruct the scene. All reconstructions are run on a machine with an NVIDIA GeForce RTX 3090 GPU with 24GB VRAM.

Scene	Real Scene	Manual Background Result	Automatic Background Result
dorm_1			
dorm_2			
dorm_3			
dorm_4			
dorm_5			

Table L.5: **Background reconstruction comparison.** The results for the real scene alongside results from the manual and automated background pipelines for five dorm scenes.

Table L.6: **Render-vs-real Background Reconstruction Metrics.** Render-vs-real agreement averaged over five scenes (dorm_1, dorm_2, dorm_3, dorm_4, dorm_5).

Variant	PSNR↑	SSIM↑	MAE↓	RMSE↓	NCC↑	$\Delta E \downarrow$	EdgeMAE↓
Manual Pipeline	12.91	0.497	0.1712	0.2294	0.549	20.13	0.0283
Automatic Pipeline	**15.29**	**0.605**	**0.1275**	**0.1758**	**0.749**	**15.23**	**0.0248**